



**POLITECNICO**  
**MILANO 1863**

SCUOLA DI INGEGNERIA INDUSTRIALE  
E DELL'INFORMAZIONE

# Automation of a Storage Warehouse

## Group 18

AUTOMATION AND CONTROL LABORATORY

Authors:

**Molettieri Luca - 11046124**

**Nicolussi Zom William - 10707926**

**Rampazzo Matteo - 10730187**

Academic Year: 2024-2025

# Contents

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>System Description</b>                         | <b>4</b>  |
| 1.1      | Warehouse Model . . . . .                         | 4         |
| 1.2      | Programmable Logic Controller (PLC) . . . . .     | 5         |
| 1.3      | Development environment . . . . .                 | 5         |
| 1.3.1    | Model structure . . . . .                         | 6         |
| 1.3.2    | Inputs . . . . .                                  | 6         |
| 1.3.3    | Outputs . . . . .                                 | 7         |
| <b>2</b> | <b>Software Architecture</b>                      | <b>8</b>  |
| 2.1      | Defined Types . . . . .                           | 8         |
| 2.2      | Global Constants . . . . .                        | 9         |
| 2.3      | Global Variables . . . . .                        | 9         |
| 2.4      | PLC_PRG - Main Program Logic . . . . .            | 11        |
| 2.5      | Initializations Function . . . . .                | 12        |
| 2.6      | CheckMode Function Block . . . . .                | 13        |
| 2.7      | CheckPosStacker Function . . . . .                | 14        |
| 2.8      | ContactsStatusGood Function . . . . .             | 14        |
| 2.9      | SetPalletValue Function Block . . . . .           | 15        |
| 2.10     | UpdateArraySensors Function . . . . .             | 15        |
| 2.11     | Movement and Position Control Functions . . . . . | 16        |
| 2.12     | Slot Management Functions . . . . .               | 19        |
| 2.13     | DropPallet SFC . . . . .                          | 22        |
| 2.14     | PickPallet SFC . . . . .                          | 23        |
| <b>3</b> | <b>Manual Mode</b>                                | <b>26</b> |
| <b>4</b> | <b>Move Pallet Mode</b>                           | <b>29</b> |
| <b>5</b> | <b>Store Pallet Mode</b>                          | <b>33</b> |
| <b>6</b> | <b>Pick Pallet Mode</b>                           | <b>38</b> |
| <b>7</b> | <b>Graphical User Interface</b>                   | <b>41</b> |
| 7.1      | Mode selection interface . . . . .                | 41        |

|           |                                      |           |
|-----------|--------------------------------------|-----------|
| 7.2       | Manual mode interface . . . . .      | 42        |
| 7.3       | Slot management interface . . . . .  | 43        |
| 7.4       | Pick pallet interface . . . . .      | 43        |
| 7.5       | Store pallet interface . . . . .     | 44        |
| 7.6       | Move pallet interface . . . . .      | 45        |
| <b>8</b>  | <b>Scalability</b>                   | <b>47</b> |
| 8.1       | Number of Rows and Columns . . . . . | 47        |
| 8.2       | Add Sensors . . . . .                | 47        |
| 8.3       | Visualization . . . . .              | 47        |
| <b>9</b>  | <b>Testing and Validation</b>        | <b>48</b> |
| <b>10</b> | <b>Conclusion</b>                    | <b>49</b> |

# Introduction

The aim of this project was to automate a model of a storage warehouse. Starting from a physical system managed by an ABB PLC, our task was to develop the control logic entirely from scratch and bring the system to operational functionality. The main objective was to design and implement an automatic management system capable of handling the movement of items in and out of storage in a reliable and flexible way.

To achieve this, we applied concepts from discrete event systems to structure the logic of a stacker crane operating in three dimensions (X, Y, Z). The system was developed in CoDeSys 2.3. The control strategy includes both manual and automated operation modes, with a focus on modularity, error handling, and scalability.

The user interface allows operators to interact with the system in a simple and safe manner, enabling manual control of the stacker as well as pallet insertion and retrieval operations. All design decisions were guided by a modular and scalable approach, with the use of constants—such as the number of rows and columns or the home position of the stacker—allowing the system to be quickly reconfigured based on new requirements.

At the foundation of the project lies the modeling of the system as a discrete event system, which ensures that the behavior of the warehouse responds precisely to specific conditions and external inputs. Error handling and state management have been carefully implemented to guarantee reliability, safety, and clear communication with the user.

# 1 | System Description

This section introduces the key physical and digital elements, along with the software environment used to control the warehouse model.

## 1.1 Warehouse Model

The warehouse model consists of a  $3 \times 3$  grid that simulates real storage slots, where a placeholder pallet can be easily positioned. Pallet movements are handled by a mechanical arm, which can freely move along the X, Y, and Z axes thanks to three DC motors. The torque generated by the motors is transmitted to the arm through a belt system. As a discrete system, both inputs and outputs are digital signals. The position of the arm is determined using multiple handkeys that act as position encoders. Additionally, the system includes a Hall effect sensor for pallet presence detection, two manual handkeys for user input, and two programmable LEDs. Beneath the model, a preassembled electrical wiring system supplies power to all components and includes built-in safety mechanisms. In particular, it automatically cuts power to the motors when the arm reaches the physical limits of the system, preventing mechanical damage. In addition, a junction box is used to connect the model to the PLC, ensuring a safe and organized wiring between the system and the controller.

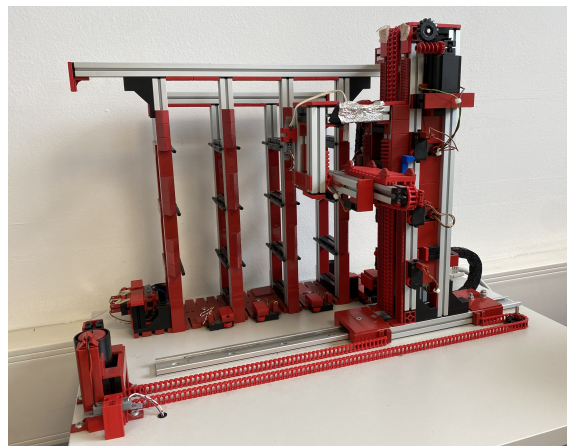


Figure 1.1: Warehouse Model

## 1.2 Programmable Logic Controller (PLC)

To automate our system, we used an ABB PM554 PLC along with the corresponding DC532 I/O module.

The PLC operates at 24V DC, features 142 kB of memory, and supports communication via both Ethernet and RS485 ports.

The external I/O module, also powered at 24V DC, provides 16 digital inputs and an additional 16 configurable ports that can function as either inputs or outputs.

## 1.3 Development environment

To program the PLC, we used the standard development environment provided by ABB: Automation Builder. This software handles the connection between the PC and the PLC and integrates with CODESYS, the tool used to develop the control logic.

CODESYS supports multiple programming languages, including Ladder Diagram (LD), Function Block Diagram (FBD), Sequential Function Chart (SFC), and Structured Text (ST) a Pascal-based language.

By connecting the Ethernet cable between the PC and the PLC, the system automatically establishes communication, allowing us to go online and begin programming.

The first step in our work was to map the I/O ports of the physical system within Automation Builder.

Based on the wiring layout of the junction box, we assigned each port of the DC532 module a custom global label. These labels are then used throughout the development process to read from or write to the corresponding signals in the system.

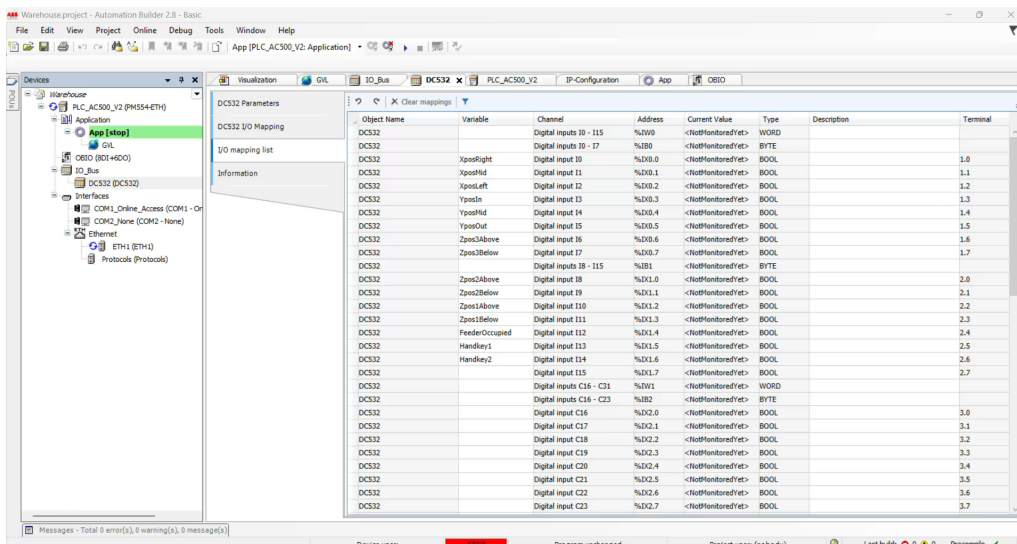


Figure 1.2: Pin Mapping

### 1.3.1 Model structure

Our PLC reads a total of 15 digital input signals.

Each handkey switches from HIGH to LOW when pressed, indicating that the mechanical arm has reached a specific position.

The inputs are organized as follows:

- **3 X-axis sensors:** Right, Middle, and Left positions
- **3 Y-axis sensors:** Load, Middle, and Drop positions
- **6 Z-axis sensors:** Bottom, Middle, and Top positions — each with an additional "Above" and "Below" level to detect when the arm is aligned over the pallet rack
- **2 User buttons:** for manual interaction
- **1 Hall-effect sensor:** for detecting the presence of a pallet

Regarding the output the PLC control 8 signal:

- **X Motor:** Direct and Reverse
- **Y Motor:** Direct and Reverse
- **Z Motor:** Direct and Reverse
- **2 LEDs:** for User Interface

### 1.3.2 Inputs

This section lists the input signals used for warehouse control and monitoring, along with their respective PLC addressing details and wire box connections.

| Name           | PLC Input | PLC Address | Wire Box | Description |
|----------------|-----------|-------------|----------|-------------|
| XposRight      | I0        | %IX0.0      | 1        | NC          |
| XposMid        | I1        | %IX0.1      | 2        | NC          |
| XposLeft       | I2        | %IX0.2      | 3        | NC          |
| YposIn         | I3        | %IX0.3      | 4        | NC          |
| YposMid        | I4        | %IX0.4      | 5        | NC          |
| YposOut        | I5        | %IX0.5      | 6        | NC          |
| Zpos3Above     | I6        | %IX0.6      | 7        | NC          |
| Zpos3Below     | I7        | %IX0.7      | 8        | NC          |
| Zpos2Above     | I8        | %IX1.0      | 9        | NC          |
| Zpos2Below     | I9        | %IX1.1      | 10       | NC          |
| Zpos1Above     | I10       | %IX1.2      | 11       | NC          |
| Zpos1Below     | I11       | %IX1.3      | 12       | NC          |
| FeederOccupied | I12       | %IX1.4      | 13       | NO          |
| Handkey1       | I13       | %IX1.5      | 14       | NO          |
| Handkey2       | I14       | %IX1.6      | 15       | NO          |

Figure 1.3: PLC inputs

### 1.3.3 Outputs

This section describes the output signals managed by the PLC to activate the warehouse components, together with their respective PLC addressing details and wire box connections.

| Name         | PLC Output | PLC Address | Wire Box | Description           |
|--------------|------------|-------------|----------|-----------------------|
| XRight       | C24        | %QX1.0      | 20       | Move motor X to right |
| XLeft        | C25        | %QX1.1      | 21       | Move motor X to left  |
| YOut         | C27        | %QX1.2      | 23       | Move motor Y out      |
| YIn          | C26        | %QX1.3      | 22       | Move motor Y in       |
| ZUp          | C28        | %QX1.4      | 24       | Move motor Z up       |
| ZDown        | C29        | %QX1.5      | 25       | Move motor Z down     |
| Led1         | C30        | %QX1.6      | 26       | Red LED               |
| Led2         | C31        | %QX1.7      | 27       | Green LED             |
| Power supply | -          | -           | 18       | -                     |
| Power supply | -          | -           | 19       | -                     |
| Power supply | -          | -           | 36       | -                     |
| Power supply | -          | -           | 37       | -                     |

Figure 1.4: PLC outputs



# 2 | Software Architecture

Our high-level software is built on top of low-level functions that handle essential operations such as axis movement, arm position checks, pick and drop basic behavior, and safety control routines.

To manage the physical system, we defined scalable constants, such as the grid size, so that the code can easily be adapted to larger warehouse configurations. To enable communication between subprograms and with the graphical interface, we implemented a set of global variables. For improved readability and maintainability, we also defined custom data structures.

At startup, the software performs an initialization routine: it checks the current position of the mechanical arm and, if unknown, moves the arm until one of the handkeys is triggered, allowing position decoding. At the same time, the data structures representing the slot states are cleared and reset.

As its main operation, the program continuously checks which working mode is selected from the graphical interface and activates the corresponding function, enabling the user to interact with the system accordingly.

## 2.1 Defined Types

In order to make the code more readable, the following **enumerations** and **structs** were declared:

```
1  TYPE ModeEnum : (  
2      stop,  
3      pause,  
4      manualControl,  (*To manually move with buttons*)  
5      movePalletTo,   (*To move pallet from source to destination*)  
6      userStore,      (*To store a pallet without select position*)  
7      userPick        (*To pick a pallet given ID*)  
8  );  END_TYPE  
9  
10 TYPE Pos2DStruct :  
11 STRUCT  
12     X: INT;    Z: INT;  
13 END_STRUCT  
14 END_TYPE  
15  
16 TYPE Pos3DStruct :  
17 STRUCT  
18     X: INT;    Y: INT;    Z: INT;
```

```

19 END_STRUCT
20 END_TYPE
21
22 TYPE SlotStruct :
23 STRUCT
24     isEmpty: BOOL; (* If a slot is empty *)
25     Row: INT; Col: INT;
26     palletID: STRING;
27 END_STRUCT
28 END_TYPE
29
30 TYPE StackerStruct :
31 STRUCT
32     X: INT; Y: INT; Z: INT;
33     isEmpty: BOOL; (* If the arm has a pallet *)
34 END_STRUCT
35 END_TYPE

```

They are defined as follows:

- **ModeEnum**: Contains the various working modes.
- **Pos2DStruct**: Describes a position with X and Y coordinates.
- **Pos3DStruct**: Describes a position with X, Y, and Z coordinates. A coordinate of 0 means unknown position.
- **SlotStruct**: Describes the position and state of each slot.
- **StackerStruct**: Describes the position and state of the arm.

## 2.2 Global Constants

To define the fundamental parameters of our warehouse system, we introduce a set of **global constants**. These constants are crucial for scaling the physical structure and managing operational logic:

```

1 VAR_GLOBAL CONSTANT
2     N_ROWS : INT := 3;
3     N_COLS : INT := 3;
4     N_SLOTS : INT := 9;
5     N_Z_SENSORS : INT := 6; (*Do 2*N_ROWS*)
6     posHome: Pos3DStruct := (X:=3, Y:=2, Z:=4); (* Home *)
7 END_VAR

```

This configuration allows for a scalable definition of the warehouse's rows and columns, along with a predefined **posHome** for all loading and unloading operations.

## 2.3 Global Variables

This section defines the **global variables** that are accessible and utilized throughout the software program. These variables are categorized according to their primary function:

```

1  VAR_GLOBAL
2      slot : ARRAY[1..N_ROWS, 1..N_COLS] OF SlotStruct;
3      stacker: stackerStruct := (X:=0, Y:=0, Z:=0);
4      Mode: ModeEnum := stop;
5      CheckMode: CheckMode;
6
7      (* --- VAR used in the visualization windows --- *)
8      ButtonMovements: ButtonMovements;
9      ButtonsMovePallets: ButtonsMovePallets;
10     CreateFlatSlotArray: CreateFlatSlotArray;
11     SetPalletValue: SetPalletValue;
12
13     (* --- Scalability sensors --- *)
14     xPosSensors: ARRAY[1..N_COLS] OF BOOL;
15     zPosSensors: ARRAY[1..N_Z_SENSORS] OF BOOL;
16 END_VAR

```

The global variables serve distinct purposes within the system:

- **Program Variables:**

- slot: An array representing each storage slot in the warehouse.
- stacker: An element defining the state and position of the mechanical arm.
- Mode: An enumeration variable indicating the current operational state of the system, initialized to ‘stop’.
- CheckMode: (Please add a brief description for this variable if possible, as it’s not explained in the original text.)

- **GUI Variables:** These variables facilitate interaction with the visualization windows and link to functions used in the Human-Machine Interface (HMI).

- ButtonMovements
- ButtonsMovePallets
- CreateFlatSlotArray
- SetPalletValue

- **Position Variables:** Used for sensing and managing the position along specific axes.

- xPosSensors: An array holding the status of X-axis position sensors, with one element per column.
- zPosSensors: An array holding the status of Z-axis position sensors, corresponding to the number of Z-sensors defined.

## 2.4 PLC\_PRG - Main Program Logic

In CoDeSys 2.3 projects, **PLC\_PRG** serves as the default Program Organization Unit (POU) that is automatically executed by the PLC runtime. In the context of this project, PLC\_PRG functions as the equivalent of the `void main()` function in traditional programming languages, acting as the primary entry point and orchestrating the entire application logic.

The program begins by declaring its local variables, followed by a snippet of the routine that implements the core control flow and operational logic of the warehouse system.

```
1 PROGRAM PLC_PRG
2 VAR
3     firstCycle: BOOL := TRUE;
4     PickAndDrop: PickAndDrop;
5     UserStorePallet: UserStorePallet;
6     UserPickPallet: UserPickPallet;
7     blinkingLed: BLINK;
8
9     (*----- Button VISU -----*)
10    tempVar: BOOL;
11
12    (* ----- Var VISU -----*)
13    contactsGoodVar: BOOL;
14 END_VAR
```

```
1 (* ----- Do this only at the beginning ----- *)
2 IF firstCycle THEN
3     Initializations();
4     firstCycle := FALSE;
5 END_IF
6
7 CheckMode();
8 (* ----- MANUAL MODE ----- *)
9 IF Mode>manualControl THEN
10    ButtonMovements();
11 END_IF
12
13 (* ----- MOVE FROM sourceSlot TO destinationSlot ----- *)
14 IF Mode=movePalletTo THEN
15    ButtonsMovePallets();
16    (* If-else contacts are correctly connected *)
17    IF ContactsStatusGood() THEN
18        PickAndDrop(sourceSlot:=ButtonsMovePallets.sourceSlot, destinationSlot:=
19            ButtonsMovePallets.destinationSlot);
20    ELSE
21        (*stop all motors *)
22        StopAllMotors();
23    END_IF;
24 END_IF
25
26 (* ----- STORE PALLET FROM USER ----- *)
27 IF Mode=userStore THEN
28    (* If-else contacts are correctly correctly *)
29    IF ContactsStatusGood() THEN
30        UserStorePallet();
31    ELSE
32        StopAllMotors();
33    END_IF;
34 END_IF
35
36 (* ----- GET PALLET FROM USER VALUE ----- *)
37 IF Mode=userPick THEN
```

```

37  (* If-else contacts are correctly connected *)
38  IF ContactsStatusGood() THEN
39      UserPickPallet();
40  ELSE
41      StopAllMotors();
42  END_IF;
43 END_IF
44
45 (* ----- USER PRESSED STOP ----- *)
46 IF Mode=stop THEN
47     StopAllMotors();
48 END_IF
49
50 (* ----- Manage Leds ----- *)
51 contactsGoodVar := ContactsStatusGood();
52 IF contactsGoodVar AND NOT(Mode=stop) THEN
53     Led1 := FALSE;
54     blinkingLed(ENABLE:= TRUE, TIMELOW:= T#500ms, TIMEHIGH:= T#500ms, OUT=>Led2);
55 ELSE
56     Led2 := FALSE;
57     blinkingLed(ENABLE:= TRUE, TIMELOW:= T#500ms, TIMEHIGH:= T#500ms, OUT=>Led1);
58 END_IF;

```

Initially, PLC\_PRG executes a one-time `Initializations()` routine. Afterward, it continuously monitors the HMI-controlled `Mode` to perform operations. A critical `ContactsStatusGood()` safety check precedes all material handling; failure immediately stops motors. Two blinking LEDs provide real-time safety status and overall system state (active or stopped).

## 2.5 Initializations Function

The **Initializations** function, executed once at system startup, is responsible for setting the initial state of key warehouse components.

```

1  FUNCTION Initializations : BOOL
2  VAR
3      i, j: INT;
4  END_VAR

```

```

1  (* ----- Initialization of the stacker ----- *)
2  (* Check if a button is pressed, otherwise, the X and Z remain 0 as in the
   initialization *)
3  CheckPosStacker();
4  UpdateArraySensors();
5
6  (* ----- Initialization of the slots as empty ----- *)
7  FOR i := 1 TO N_ROWS DO
8      FOR j := 1 TO N_COLS DO
9          slot[i,j].Row := i;
10         slot[i,j].Col := j;
11         ClearSlot(slot[i,j]);
12     END_FOR
13 END_FOR

```

This function primarily performs three initial setup tasks: it determines and sets the stacker's initial position (`CheckPosStacker` and `UpdateArraySensors`), and it initializes all storage slots within the warehouse grid as empty, assigning their respective row and column indices.

## 2.6 CheckMode Function Block

The **CheckMode** function block is responsible for managing the warehouse's operational mode based on user input from the Human-Machine Interface (HMI) and critical physical safety signals.

```
1 FUNCTION_BLOCK CheckMode
2
3 VAR
4   (* Variables connected to the GUI buttons *)
5   buttonStopPressed: BOOL;
6   buttonHomePressed: BOOL;
7   buttonManualPressed: BOOL;
8   buttonPalletToPressed: BOOL;
9   buttonUserStorePressed: BOOL;
10  buttonUserStoreGoPressed: BOOL;
11  buttonUserPickPressed: BOOL;
12  buttonUserPickGoPressed: BOOL;
13  buttonUserPickPopUpPressed: BOOL;
14  buttonUserPickPopUpStopPressed: BOOL;
15  buttonPopupStorePalletIDContinuePressed: BOOL;
16  buttonPopupStorePalletIDStopPressed: BOOL;
17  buttonPopupSlotsFullPressed: BOOL;
18
19
20  (* Vars for button pressed *)
21  tHoldHandkey2 : TON;
22  prevHandkey2 : BOOL;
23 END_VAR
```

```
1 (* Check if button is pressed *)
2 IF buttonStopPressed THEN
3   Mode := stop;
4 ELSIF buttonManualPressed THEN
5   Mode := manualControl;
6 ELSIF buttonPalletToPressed THEN
7   Mode := movePalletTo;
8 ELSIF buttonUserStorePressed THEN
9   Mode := userStore;
10 ELSIF buttonHomePressed THEN
11  Mode := pause;
12 ELSIF buttonUserPickPressed THEN
13  Mode := userPick;
14 END_IF
15 (* If physical button Handkey2 is pressed, stop the system *)
16 IF Handkey2 THEN
17   Mode := stop;
18 END_IF
19 (* Measure pressed of Handkey2 and act accordingly *)
20 IF Mode=stop THEN
21   tHoldHandkey2(IN := Handkey2, PT := T#5s);
22 ELSE
23   tHoldHandkey2(IN := FALSE);
24 END_IF;
25 IF tHoldHandkey2.Q THEN
26   Mode := pause;
27 END_IF;
```

The function block operates by evaluating activated HMI buttons, dynamically setting the system's Mode accordingly. Additionally, it incorporates logic to monitor the 'Handkey2' physical input: a brief press stops the system, while a sustained press for 5 seconds (managed by the 'TON' timer 'tHoldHandkey2') transitions the system into a 'pause' state, enabling robust safety and operational control.

## 2.7 CheckPosStacker Function

The **CheckPosStacker** function is dedicated to determining and updating the current spatial coordinates of the mechanical arm (stacker) within the warehouse.

```
1 FUNCTION CheckPosStacker : BOOL
2 VAR
3     i: INT;
4 END_VAR

1 (* ----- Initialization of the stacker ----- *)
2 FOR i := 1 TO N_COLS DO
3     IF CheckPosX(posToCheck:=i)=TRUE THEN stacker.X:=i; END_IF
4 END_FOR
5
6 FOR i:=1 TO 3 DO
7     IF CheckPosY(posToCheck:=i)= TRUE THEN stacker.Y:=i; END_IF
8 END_FOR
9
10 FOR i := 1 TO N_Z_SENSORS DO
11     IF CheckPosZ(posToCheck:=i)=TRUE THEN stacker.Z:=i; END_IF
12 END_FOR
```

This function iteratively reads the status of position sensors along the X, Y, and Z axes, subsequently assigning the detected coordinates to the respective X, Y, and Z fields of the global **stacker** object.

## 2.8 ContactsStatusGood Function

The **ContactsStatusGood** function is a crucial safety mechanism designed to prevent malfunctions caused by erroneous sensor readings, specifically when the PLC's contact box experiences disconnections.

```
1 FUNCTION ContactsStatusGood : BOOL
2 VAR
3     i : INT;
4     sumX: INT:=0;    sumY: INT:=0;    sumZ: INT:=0;
5 END_VAR

1 (* ----- Sometimes the PLC disconnects to the pins ----- *)
2 (* Sum the booleans and if something is wrong, block *)
3
4 FOR i:=1 TO N_COLS DO
5     IF CheckPosX(posToCheck:=i) THEN
6         sumX:=sumX+1;
7     END_IF
8 END_FOR
9
10 FOR i:=1 TO 3 DO
11     IF CheckPosY(posToCheck:=i) THEN
12         sumY:=sumY+1;
13     END_IF
14 END_FOR
15
16 FOR i:=1 TO N_Z_SENSORS DO
17     IF CheckPosZ(posToCheck:=i) THEN
```

```

18     sumZ := sumZ+1;
19     END_IF
20 END_FOR
21
22 (* --- Check that no more than one sensor is pressed --- *)
23 IF sumX>1 OR sumY>1 OR sumZ>1 THEN
24     ContactsStatusGood := FALSE;
25 ELSE
26     ContactsStatusGood := TRUE;
27 END_IF

```

This function validates sensor integrity by ensuring that no more than one position sensor is active simultaneously along the X, Y, or Z axis. If multiple sensors are detected as active on any single axis, the function returns **FALSE**, signaling an error state that blocks further operations to maintain system safety.

## 2.9 SetPalletValue Function Block

The **SetPalletValue** function block is designed to facilitate user input for pallet identification.

```

1 FUNCTION_BLOCK SetPalletValue
2 VAR_INPUT
3 END_VAR
4 VAR_OUTPUT
5 END_VAR
6 VAR
7     str: STRING; (* Internal variable to store the entered string *)
8 END_VAR

```

This function block serves as an interface for the Human-Machine Interface (HMI), allowing operators to input or modify the string (e.g., ID) associated with a pallet via the GUI's keypad. Its primary role is data acquisition from the user interface, without performing direct physical actions.

## 2.10 UpdateArraySensors Function

The **UpdateArraySensors** function is responsible for populating the global sensor arrays with real-time input states.

```

1 FUNCTION UpdateArraySensors : BOOL
2
3 (* ----- Static assignement of the xPosSensors array ----- *)
4 xPosSensors[1] := XposLeft;
5 xPosSensors[2] := XposMid;
6 xPosSensors[3] := XposRight;
7
8 (* ----- Static assignement of the zPosSensors array ----- *)
9 zPosSensors[1] := Zpos1Below;
10 zPosSensors[2] := Zpos1Above;
11 zPosSensors[3] := Zpos2Below;
12 zPosSensors[4] := Zpos2Above;
13 zPosSensors[5] := Zpos3Below;
14 zPosSensors[6] := Zpos3Above;

```



This function directly maps the individual digital inputs (XposLeft, Zpos1Below, etc.) to their respective elements within the `xPosSensors` and `zPosSensors` arrays. This function intentionally utilizes a direct, static assignment to map each physical sensor input to its corresponding array element. This design decision is crucial given that CoDeSys 2.3 restricts the use of pointers to inputs. Therefore, users must manually update this function when extending the warehouse's dimensions (rows or columns) to ensure precise and ordered integration of new sensor inputs.

## 2.11 Movement and Position Control Functions

This section details the low-level functions responsible for controlling the mechanical arm's movement along its axes and verifying its position. Each function is designed to ensure safe and precise operation within the warehouse.

### Check Position Functions (CheckPosX, CheckPosY, CheckPosZ)

These functions return `TRUE` if the mechanical arm is currently at the specified input position along its respective axis (X, Y, or Z). They achieve this by checking the status of corresponding physical position sensors. For `CheckPosX` and `CheckPosZ`, the global sensor arrays (`xPosSensors`, `zPosSensors`) are first updated.

```

1 FUNCTION CheckPosX : BOOL
2 VAR_INPUT
3     posToCheck: INT;
4 END_VAR
5 VAR
6     buttonPressed: BOOL := FALSE;
7 END_VAR
8 UpdateArraySensors();
9 IF xPosSensors[posToCheck]=FALSE THEN
10     buttonPressed := TRUE;
11 END_IF
12 CheckPosX := buttonPressed;

```

```

1 FUNCTION CheckPosY : BOOL
2 VAR_INPUT
3     posToCheck: INT;
4 END_VAR
5 VAR
6     buttonPressed: BOOL := FALSE;
7 END_VAR
8 CASE posToCheck OF
9     1: IF YposIn=FALSE THEN buttonPressed:=TRUE; END_IF
10     2: IF YposMid=FALSE THEN buttonPressed:=TRUE; END_IF
11     3: IF YposOut=FALSE THEN buttonPressed:=TRUE; END_IF
12 END_CASE
13 CheckPosY := buttonPressed;

```

```

1 FUNCTION CheckPosZ : BOOL
2 VAR_INPUT
3     posToCheck: INT;
4 END_VAR
5 VAR

```

```

6   buttonPressed: BOOL := FALSE;
7   END_VAR
8   UpdateArraySensors();
9   IF zPosSensors[posToCheck]=FALSE THEN
10      buttonPressed := TRUE;
11   END_IF
12   CheckPosZ := buttonPressed;

```

## Axis Movement Functions (MoveX, MoveY, MoveZ)

These functions command the mechanical arm to move along a single axis (X, Y, or Z) towards a specified destination. They return TRUE while the arm is moving and FALSE once the destination is reached and verified by the corresponding CheckPos function. All movements are contingent upon a successful ContactsStatusGood() safety check and CheckPosStacker() for current position update. Importantly, MoveX and MoveZ operations are permissible only when the arm is outside the storage grid (specifically `stacker.Y > 1`), preventing collisions.

```

1   FUNCTION MoveX : BOOL
2   VAR_INPUT
3       destinationX: INT;
4   END_VAR
5   CheckPosStacker();
6   IF ContactsStatusGood() THEN
7       IF stacker.Y > 1 THEN
8           IF stacker.X < destinationX THEN
9               XRight := TRUE; XLeft := FALSE;
10              ELIF stacker.X > destinationX OR NOT(CheckPosX(posToCheck:=destinationX)) THEN
11                  XRight := FALSE; XLeft := TRUE;
12              ELSE XRight := FALSE; XLeft := FALSE;
13              END_IF
14          ELSE XRight := FALSE; XLeft := FALSE;
15          END_IF
16          IF stacker.X=destinationX AND CheckPosX(posToCheck:=destinationX) THEN
17              MoveX := FALSE;
18          ELSE MoveX := TRUE;
19          END_IF
20      END_IF

```

```

1   FUNCTION MoveY : BOOL
2   VAR_INPUT
3       destinationY: INT;
4   END_VAR
5   CheckPosStacker();
6   IF ContactsStatusGood() THEN
7       IF stacker.Y < destinationY THEN
8           YIn := FALSE; YOut := TRUE;
9       ELIF stacker.Y > destinationY OR NOT( CheckPosY(posToCheck:=destinationY)) THEN
10          YIn := TRUE; YOut := FALSE;
11       ELSE YIn := FALSE; YOut := FALSE;
12       END_IF
13       IF stacker.Y=destinationY AND CheckPosY(posToCheck:=destinationY) THEN
14          MoveY := FALSE;
15       ELSE MoveY := TRUE;
16       END_IF
17   END_IF

```

```

1 FUNCTION MoveZ : BOOL
2 VAR_INPUT
3     destinationZ: INT;
4 END_VAR
5 CheckPosStacker();
6 IF ContactsStatusGood() THEN
7     IF stacker.Z < destinationZ THEN
8         ZUp := TRUE; ZDown := FALSE;
9     ELSIF stacker.Z > destinationZ OR NOT(CheckPosZ(posToCheck:=destinationZ)) THEN
10        ZUp := FALSE; ZDown := TRUE;
11    ELSE ZUp := FALSE; ZDown := FALSE;
12    END_IF
13    IF stacker.Z=destinationZ AND CheckPosZ(posToCheck:=destinationZ) THEN
14        MoveZ := FALSE;
15    ELSE MoveZ := TRUE;
16    END_IF
17 END_IF

```

## Composite Movement Functions (MoveXZ, MoveHome)

These functions orchestrate movements across multiple axes or to predefined locations.

- MoveXZ: Moves the arm in sync along the X and Z axes to destinationPos. This composite movement is also restricted to when the arm is outside the storage grid (stacker.Y > 1).
- MoveHome: Commands the mechanical arm to move to the posHome (predefined home) position. It returns TRUE while any axis is still in motion.

```

1 FUNCTION MoveXZ : BOOL
2 VAR_INPUT
3     destinationPos: Pos2DStruct;
4 END_VAR
5 CheckPosStacker();
6 IF ContactsStatusGood() THEN
7     IF stacker.Y > 1 THEN
8         MoveX(destinationPos.X);
9         YIn := FALSE; YOut:=FALSE;
10        MoveZ(destinationPos.Z);
11    ELSE
12        XRight := FALSE; XLeft := FALSE;
13        ZUp := FALSE; ZDown := FALSE;
14        MoveY(2); (*MoveY(yMid);*)
15    END_IF
16    IF stacker.X=destinationPos.X AND stacker.Z=destinationPos.Z THEN
17        MoveXZ := FALSE;
18    ELSE MoveXZ := TRUE;
19    END_IF
20 END_IF

```

```

1 FUNCTION MoveHome : BOOL
2 VAR
3     motorsMoving: BOOL := FALSE;
4 END_VAR
5 CheckPosStacker();
6 IF ContactsStatusGood() THEN
7     IF MoveX(destinationX:=posHome.X) THEN motorsMoving := TRUE; END_IF
8     IF MoveY(destinationY:=posHome.Y) THEN motorsMoving := TRUE; END_IF
9     IF MoveZ(destinationZ:=posHome.Z) THEN motorsMoving := TRUE; END_IF
10    MoveHome := motorsMoving;
11 END_IF

```

## StopAllMotors Function

This function immediately deactivates all motor contactors, bringing all mechanical arm movements to a halt. It is primarily invoked as a safety measure upon detection of errors or system stoppage.

```
1 FUNCTION StopAllMotors : BOOL
2 XRight := FALSE;
3 XLeft := FALSE;
4 YIn := FALSE;
5 YOut := FALSE;
6 ZUp := FALSE;
7 ZDown := FALSE;
```

## 2.12 Slot Management Functions

This section details the functions responsible for managing the state and content of the warehouse's storage slots, including operations for clearing, copying, searching, and identifying available locations.

### ClearSlot

This function sets a specified `slotToClear` as empty and clears its `palletID`.

```
1 FUNCTION ClearSlot : BOOL
2 VAR_INPUT
3     slotToClear: SlotStruct;
4 END_VAR
5 slot[slotToClear.Row, slotToClear.Col].isEmpty := TRUE;
6 slot[slotToClear.Row, slotToClear.Col].palletID := '';
```

### CopySlotToAnother

This function copies the `palletID` from `slotOld` to `slotNew`, marking `slotNew` as occupied.

```
1 FUNCTION CopySlotToAnother : BOOL
2 VAR_INPUT
3     slotOld: SlotStruct;
4     slotNew: SlotStruct;
5 END_VAR
6 slot[slotNew.Row, slotNew.Col].isEmpty := FALSE;
7 slot[slotNew.Row, slotNew.Col].palletID := slot[slotOld.Row, slotOld.Col].palletID;
```

### CreateFlatSlotArray

This function block generates a one-dimensional array (`slotFlat`) from the two-dimensional slot array for visualization in the table instruments in the HMI.

```
1 FUNCTION_BLOCK CreateFlatSlotArray
2 VAR
3     i, j : INT;
4     slotFlat : ARRAY[1..N_SLOTS] OF SlotStruct;
```

```

5  END_VAR
6  FOR i := 1 TO N_ROWS DO
7      FOR j := 1 TO N_COLS DO
8          slotFlat[(i-1)*N_COLS + j] := slot[i, j];
9      END_FOR
10 END_FOR

```

## GetEmptySlot

This function identifies and returns the **SlotStruct** of the nearest available empty slot within the warehouse. It employs an algorithm based on the **Manhattan distance** from the **posHome** location, systematically expanding its search outwards. This ensures that priority is given to slots closest to the loading/unloading point, minimizing travel time.

```

1  FUNCTION GetEmptySlot : SlotStruct
2  VAR
3      i, j: INT;
4      startingSlot: SlotStruct;
5      distanceHome: INT;
6      maxDist: INT;
7      deltaR, deltaC: INT;
8      newR, newC: INT;
9  END_VAR
10 (* First, it is needed to convert from home pos (X,Y,Z) to slot pos (Row,Col) *)
11 startingSlot.Row := posHome.X;
12 startingSlot.Col := (posHome.Z+1) / 2;
13 (* Then find the nearest available slot according to Manhattan distance *)
14 distanceHome := 0;
15 maxDist := MAX(startingSlot.Row - 1, N_ROWS - startingSlot.Row) + MAX(startingSlot.
    Col - 1, N_COLS - startingSlot.Col);
16
17 IF NOT SlotsFull() THEN
18     WHILE distanceHome <= maxDist DO
19         FOR deltaR := -distanceHome TO distanceHome DO
20             deltaC := distanceHome - ABS(deltaR);
21
22             newR := startingSlot.Row + deltaR;
23             newC := startingSlot.Col + deltaC;
24
25             IF (newR >= 1) AND (newR <= N_ROWS) AND (newC >= 1) AND (newC <= N_COLS) THEN
26                 IF slot[newR, newC].isEmpty THEN
27                     GetEmptySlot := slot[newR, newC];
28                     RETURN;
29                 END_IF
30             END_IF
31
32             IF deltaC <> 0 THEN
33                 newR := startingSlot.Row + deltaR;
34                 newC := startingSlot.Col - deltaC;
35
36                 IF (newR >= 1) AND (newR <= N_ROWS) AND (newC >= 1) AND (newC <= N_COLS)
37                     THEN
38                     IF slot[newR, newC].isEmpty THEN
39                         GetEmptySlot := slot[newR, newC];
40                         RETURN;
41                     END_IF
42                 END_IF
43             END_IF
44             distanceHome := distanceHome + 1;
45         END_FOR
46     END_WHILE
47 END_IF

```

## GetSlotFromPalletID

This function searches the warehouse slots for a specific `strPalletID` and returns the `SlotStruct` where the pallet is found.

```
1 FUNCTION GetSlotFromPalletID : SlotStruct
2 VAR_INPUT
3     strPalletID: STRING;
4 END_VAR
5 VAR
6     i, j: INT;
7 END_VAR
8 FOR i := 1 TO N_ROWS DO
9     FOR j := 1 TO N_COLS DO
10        IF NOT slot[i,j].isEmpty THEN
11            IF slot[i, j].palletID = strPalletID THEN
12                GetSlotFromPalletID := slot[i,j];
13                RETURN;
14            END_IF
15        END_IF
16    END_FOR
17 END_FOR
```

## PalletIDInStock

This function checks if a pallet with the given `strPalletID` is currently stored within the warehouse, returning `TRUE` if found, otherwise `FALSE`.

```
1 FUNCTION PalletIDInStock : BOOL
2 VAR_INPUT
3     strPalletID: STRING;
4 END_VAR
5 VAR
6     i, j: INT;
7 END_VAR
8 FOR i := 1 TO N_ROWS DO
9     FOR j := 1 TO N_COLS DO
10        IF NOT slot[i,j].isEmpty THEN
11            IF slot[i, j].palletID = strPalletID THEN
12                PalletIDInStock := TRUE;
13                RETURN;
14            END_IF
15        END_IF
16    END_FOR
17 END_FOR
18 PalletIDInStock := FALSE;
```

## SlotsFull

This function determines if all storage slots in the warehouse are currently occupied, returning `TRUE` if no empty slot is found, otherwise `FALSE`.

```
1 FUNCTION SlotsFull : BOOL
2 VAR
3     i, j: INT;
4 END_VAR
5 FOR i:=1 TO N_ROWS DO
6     FOR j := 1 TO N_COLS DO
7         IF slot[i, j].isEmpty THEN
8             SlotsFull := FALSE;
```

```

9      RETURN;
10     END_IF
11   END_FOR
12 END_FOR
13 SlotsFull := TRUE;

```

## 2.13 DropPallet SFC

The **DropPallet** function block orchestrates the sequence of movements required to place a pallet into a storage slot. This SFC (Sequential Function Chart) is designed as a sub-routine, called by higher-level "mother functions" that manage the overall pallet handling logic.

```

1 FUNCTION_BLOCK DropPallet
2 VAR_OUTPUT
3   finished: BOOL := FALSE;
4 END_VAR
5 VAR
6   initializationsDone: BOOL := FALSE;
7   state: INT := 0;
8   startZ: INT;
9 END_VAR

```

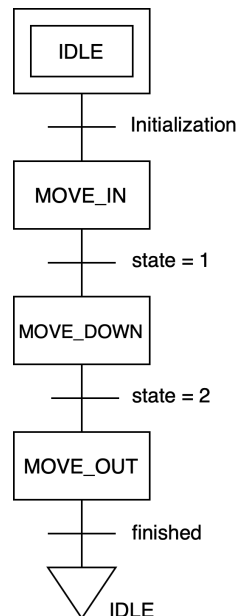


Figure 2.1: DropPallet SFC

The SFC transitions through distinct states:

- **IDLE**: Initializes the function, setting **startZ** to the stacker's current Z-position and resetting **finished**.

```

1 (* ----- IDLE ----- *)
2 startZ := stacker.Z;
3 finished:=FALSE;
4 initializationsDone := TRUE;

```

- **MOVE\_IN**: Moves the stacker inward (MoveY(1)). Upon completion, it transitions to **state = 1**.

```

1  (* ----- MOVE_IN ----- *)
2  initializationsDone := FALSE;
3  IF NOT MoveY(1) THEN
4      state := 1;
5  END_IF

```

- **MOVE\_DOWN**: Lowers the stacker by one unit from startZ (MoveZ(startZ-1)). Once completed, it transitions to **state = 2**.

```

1  (* ----- MOVE_DOWN ----- *)
2  IF NOT MoveZ(startZ-1) THEN
3      state := 2;
4  END_IF

```

- **MOVE\_OUT**: Retracts the stacker outwards (MoveY(2)). After this movement, the **finished** output is set to **TRUE**, signifying the completion of the pallet drop operation.

```

1  (* ----- MOVE_OUT ----- *)
2  IF NOT MoveY(2) THEN
3      state := 3;
4      finished := TRUE;
5  END_IF

```

## 2.14 PickPallet SFC

The **PickPallet** function block orchestrates the sequence of movements and checks required to retrieve a pallet from a storage slot. Like **DropPallet**, this SFC (Sequential Function Chart) serves as a sub-routine, invoked by higher-level "mother functions" that manage the overall pallet handling process.

```

1  FUNCTION_BLOCK PickPallet
2  VAR_OUTPUT
3      finished: BOOL := FALSE;
4  END_VAR
5  VAR
6      initializationsDone: BOOL := FALSE;
7      showPopUp: BOOL;
8      popUpClicked: BOOL;
9      state: INT := 0;
10     startZ: INT;
11     numberFeederOccupiedCounter: INT := 0;
12     palletDetectedInStacker: BOOL;
13     hasReachedY0: BOOL;
14 END_VAR

```

The SFC defines the following operational states:

- **IDLE**: Initializes the function, setting **startZ** to the stacker's current Z-position, resetting control flags (**finished**, **palletDetectedInStacker**, **showPopUp**, **popUpClicked**), and initializing the **numberFeederOccupiedCounter**.



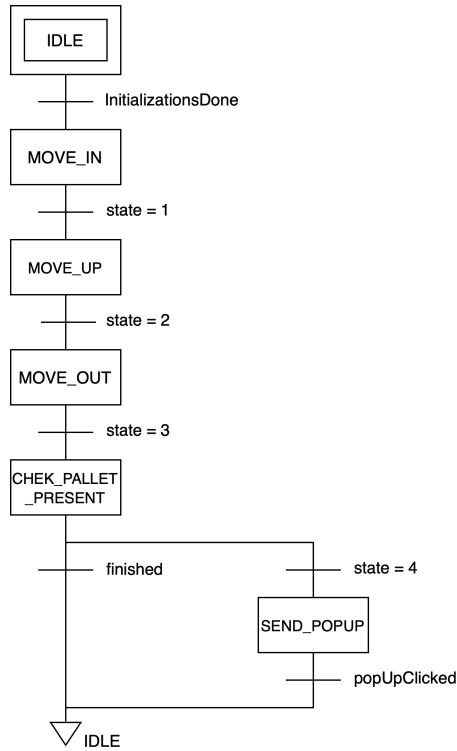


Figure 2.2: PickPallet SFC

```

1  (* ----- IDLE ----- *)
2  startZ := stacker.Z;
3  finished:=FALSE;
4  palletDetectedInStacker := FALSE;
5  showPopUp := FALSE;
6  popUpClicked := FALSE;
7  initializationsDone := TRUE;
8  numberFeederOccupiedCounter := 0;

```

- **MOVE\_IN**: Moves the stacker inward (MoveY(1)). Once at the inner position, it transitions to **state = 1**.

```

1  (* ----- MOVE_IN ----- *)
2  initializationsDone := FALSE;
3  IF NOT MoveY(1) THEN
4      state := 1;
5  END_IF

```

- **MOVE\_UP**: Raises the stacker by one unit from startZ (MoveZ(startZ+1)). Upon completion, it transitions to **state = 2**.

```

1  (* ----- MOVE_UP ----- *)
2  IF NOT MoveZ(startZ+1) THEN
3      state := 2;
4  END_IF

```

- **MOVE\_OUT**: Retracts the stacker outwards (MoveY(2)). After this movement, it transitions to **state = 3**.

```

1  (* ----- MOVE_OUT ----- *)
2  IF NOT MoveY(2) THEN
3      state := 3;
4  END_IF

```

- **CHECK\_PALLET\_PRESENT**: Attempts to detect a pallet. The stacker performs up to two checks (moving to y0 and back to yM) to verify if FeederOccupied is active. If a pallet is detected, palletDetectedInStacker is set to TRUE, motors stop, and finished is set to TRUE, returning to IDLE. If no pallet is detected after two attempts, the system proceeds to **state = 4** (SEND\_POPUP).

```

1  (* ----- CHECK_PALLET_PRESENT ----- *)
2  IF numberFeederOccupiedCounter < 2 THEN
3      IF stacker.Y = 2 THEN MoveY(3); hasReachedY0 := FALSE;
4      ELSIF stacker.Y = 3 THEN
5          IF NOT hasReachedY0 THEN numberFeederOccupiedCounter :=
6              numberFeederOccupiedCounter + 1; hasReachedY0 := TRUE; END_IF
7          MoveY(2);
8          END_IF
9          IF FeederOccupied THEN
10             palletDetectedInStacker := TRUE;
11             YIn := FALSE; Y0out := FALSE;
12             finished := TRUE;
13             END_IF
14         ELSE
15             IF NOT MoveY(2) THEN
16                 YIn := FALSE; Y0out := FALSE;
17                 state := 4;
18             END_IF
19         END_IF

```

- **SEND\_POPUP**: Activated if CHECK\_PALLET\_PRESENT fails to detect a pallet. This state displays a pop-up (showPopUp := TRUE) to the user. The function then waits for user interaction (via CheckMode.buttonUserPickPopUpPressed or CheckMode.buttonUserPickPopUpStopPressed). Upon user input (popUpClicked), showPopUp is reset, and the function concludes, potentially setting finished to TRUE or FALSE based on the button pressed.

```

1  (* ----- SEND_POPUP ----- *)
2  showPopUp := TRUE;
3  IF NOT MoveZ(startZ) THEN (* Back to startZ while waiting for popup
4      interaction *)
5      IF CheckMode.buttonUserPickPopUpPressed THEN showPopUp := FALSE; finished
6          := TRUE; popUpClicked := TRUE; END_IF
7      IF CheckMode.buttonUserPickPopUpStopPressed THEN showPopUp := FALSE;
8          finished := FALSE; popUpClicked := TRUE; END_IF
9  END_IF

```

## 3 | Manual Mode

Manual Mode provides operators with direct, granular control over the mechanical arm's movement along all three axes (X, Y, Z). This functionality is invaluable for debugging, calibration, or immediate intervention in unforeseen system states.

As illustrated in Figure 3.1, the movement logic is continuously checked against safety limits to ensure safe operation.

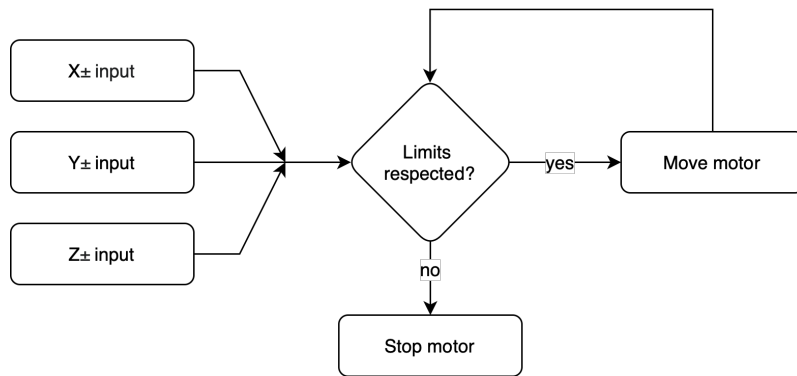


Figure 3.1: Manual mode flowchart

The core implementation for this mode resides within the `ButtonMovements` function block, directly referenced in `PLC_PRG`.

```
1 FUNCTION_BLOCK ButtonMovements
2 VAR
3   (* buttons moving the stacker *)
4   buttonRightPressed: BOOL := FALSE;
5   buttonLeftPressed: BOOL := FALSE;
6   buttonInPressed: BOOL := FALSE;
7   buttonOutPressed: BOOL := FALSE;
8   buttonUpPressed: BOOL := FALSE;
9   buttonDownPressed: BOOL := FALSE;
10
11   (* buttons insert and remove *)
12   buttonInsertPressed: BOOL := FALSE;
13   oldButtonInsertPressed: BOOL := FALSE; (*Needed to detect rising edge*)
14   buttonRemovePressed: BOOL := FALSE;
15   oldButtonRemovePressed: BOOL := FALSE; (*Needed to detect rising edge*)
16   keypadR: INT;
17   keypadC: INT;
18   buttonPopupInsertOccupied: BOOL := FALSE;
19   showPopupInsertOccupied: BOOL := FALSE;
20   buttonPopupRowColEmpty: BOOL := FALSE;
```

```

21 showPopupRowColEmpty: BOOL := FALSE;
22 buttonPopupRemoveValue: BOOL := FALSE;
23 showPopupRemoveValue: BOOL := FALSE;
24 buttonPopupStringEmpty: BOOL := FALSE;
25 showPopupStringEmpty: BOOL := FALSE;
26 END_VAR

```

```

1  (* Move only if contacts are connected *)
2  IF ContactsStatusGood() THEN
3      CheckPosStacker();
4
5      (* Move in X axis *)
6      IF buttonRightPressed AND (NOT CheckPosX(posToCheck:=N_COLS)) THEN
7          XRight := TRUE;
8          (*XRight := TRUE;
9          XLeft := FALSE;*)
10     ELSIF buttonLeftPressed AND (NOT CheckPosX(posToCheck:=1)) THEN
11         XRight := FALSE;
12         XLeft := TRUE;
13     ELSE
14         XRight := FALSE;
15         XLeft := FALSE;
16     END_IF
17
18     (* Move in Y axis *)
19     IF buttonOutPressed AND (NOT CheckPosY(posToCheck:=3)) THEN
20         YIn := FALSE;
21         YOut := TRUE;
22     ELSIF buttonInPressed AND (NOT CheckPosY(posToCheck:=1)) THEN
23         YIn := TRUE;
24         YOut := FALSE;
25     ELSE
26         YIn := FALSE;
27         YOut := FALSE;
28     END_IF
29
30     (* Move in Z axis *)
31     IF buttonUpPressed AND (NOT CheckPosZ(posToCheck:=N_Z_SENSORS)) THEN
32         ZUp := TRUE;
33         ZDown := FALSE;
34     ELSIF buttonDownPressed AND (NOT CheckPosZ(posToCheck:=1)) THEN
35         ZUp := FALSE;
36         ZDown := TRUE;
37     ELSE
38         ZUp := FALSE;
39         ZDown := FALSE;
40     END_IF
41 ELSE
42     StopAllMotors();
43 END_IF
44
45 (* --- This part manage insert / remove in the manual interface. --- *)
46
47 (* --- MANUAL INSERT ---*)
48 CreateFlatSlotArray();
49 IF buttonInsertPressed AND (NOT oldButtonInsertPressed) THEN (*IF (rising edge
50     detected)*)
51     IF (NOT (SetPalletValue.str='')) THEN
52         IF keypadR<>0 AND keypadC<>0 THEN
53             IF slot[keypadR, keypadC].isEmpty THEN
54                 slot[keypadR, keypadC].isEmpty := FALSE;
55                 slot[keypadR, keypadC].palletID := SetPalletValue.str;
56             ELSE
57                 (* show popup slot is occupied *)
58                 showPopupInsertOccupied := TRUE;
59             END_IF
60         ELSE
59             showPopupRowColEmpty := TRUE;

```

```

61     END_IF
62 ELSE
63     showPopupStringEmpty := TRUE;
64 END_IF
65 END_IF
66 oldButtonInsertPressed := buttonInsertPressed;
67
68 (* --- MANUAL REMOVE ---*)
69 IF buttonRemovePressed AND (NOT oldButtonRemovePressed) THEN (*IF (rising edge
    detected)*)
70     IF (NOT (SetPalletValue.str='')) THEN
71         IF PalletIDInStock(strPalletID:= SetPalletValue.str) THEN
72             ClearSlot(slotToClear:=GetSlotFromPalletID(strPalletID:=SetPalletValue.str));
73         ELSE
74             showPopupRemoveValue:=TRUE;
75         END_IF
76     ELSE
77         showPopupStringEmpty := TRUE;
78     END_IF
79 END_IF
80 oldButtonRemovePressed := buttonRemovePressed;
81
82 (* Manage Popup *)
83 IF buttonPopupInsertOccupied THEN
84     showPopupInsertOccupied := FALSE;
85 END_IF
86 IF buttonPopupRowColEmpty THEN
87     showPopupRowColEmpty := FALSE;
88 END_IF
89 IF buttonPopupRemoveValue THEN
90     showPopupRemoveValue := FALSE;
91 END_IF
92 IF buttonPopupStringEmpty THEN
93     showPopupStringEmpty := FALSE;
94 END_IF

```

The **ButtonMovements** function block handles two primary sets of operations. Firstly, it provides continuous movement control for the X, Y, and Z axes based on HMI button presses, always ensuring **ContactsStatusGood()** for safety and invoking **StopAllMotors()** on error. Secondly, it manages manual pallet insertion and removal operations, including validation of input data (pallet ID, target slot) and displaying appropriate pop-up messages for user feedback. Note that the system's row and column limits for manual insertion/removal are enforced via the HMI keypad.

## 4 | Move Pallet Mode

The Move Pallet Mode enables the automated relocation of a pallet from a specified `sourceSlot` to a `destinationSlot` within the warehouse grid. The system first validates input positions, ensuring a pallet is present at the source and the destination is empty. Upon successful verification, the mechanical arm autonomously moves to the source, picks up the pallet, transports it, and drops it at the destination.

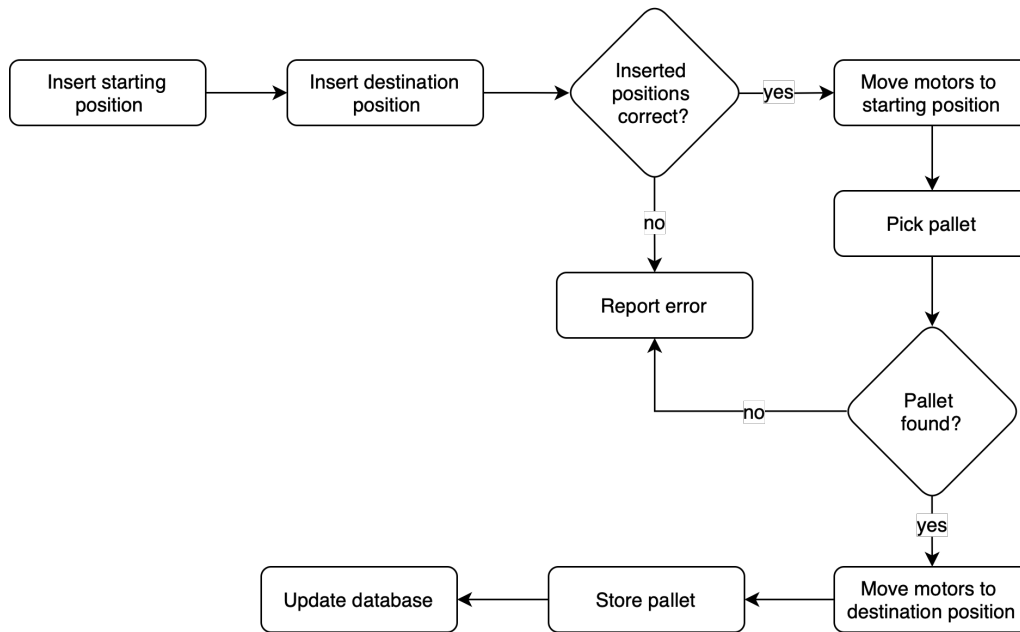


Figure 4.1: Move Pallet Mode Flowchart

This function block manages user input for the Move Pallet Mode: it handle the definition of source and destination slots via HMI keypads.

```

1 FUNCTION_BLOCK ButtonsMovePallets
2 VAR
3     sourceSlot: SlotStruct;
4     destinationSlot: SlotStruct;
5
6     (* Source and destination keypad *)
7     keypadSourceR: INT;
8     keypadSourceC: INT;
9     keypadDestinationR: INT;
10    keypadDestinationC: INT;
  
```

```

11
12     (* Button GO *)
13     btnGoPressed: BOOL;
14 END_VAR

```

```

1  (* ----- Set source position ----- *)
2  sourceSlot.Row:=keypadSourceR;
3  sourceSlot.Col:=keypadSourceC;
4
5  (* ----- Set destination position ----- *)
6  destinationSlot.Row:=keypadDestinationR;
7  destinationSlot.Col:=keypadDestinationC;

```

The overarching process, depicted in Figure 4.1, is managed by the `PickAndDrop` function block, which orchestrates the entire sequence.

```

1  FUNCTION_BLOCK PickAndDrop
2  VAR_INPUT
3      sourceSlot: SlotStruct;
4      destinationSlot: SlotStruct;
5  END_VAR
6  VAR_OUTPUT
7      finished: BOOL := FALSE;
8  END_VAR
9  VAR
10     (* Variables to change the state *)
11     showPopUp: BOOL;
12     movementPossible: BOOL;
13     sourceReached: BOOL := FALSE;
14     destinationReached: BOOL := FALSE;
15     (* Variables to store the position *)
16     sourcePos: Pos2DStruct;
17     destinationPos: Pos2DStruct;
18     (* Variables to modify if slot is empty or filled *)
19     PickPallet: PickPallet;
20     DropPallet: DropPallet;
21 END_VAR

```

The internal states and transitions of the `PickAndDrop` function block are defined by its SFC, as shown in Figure 4.2.

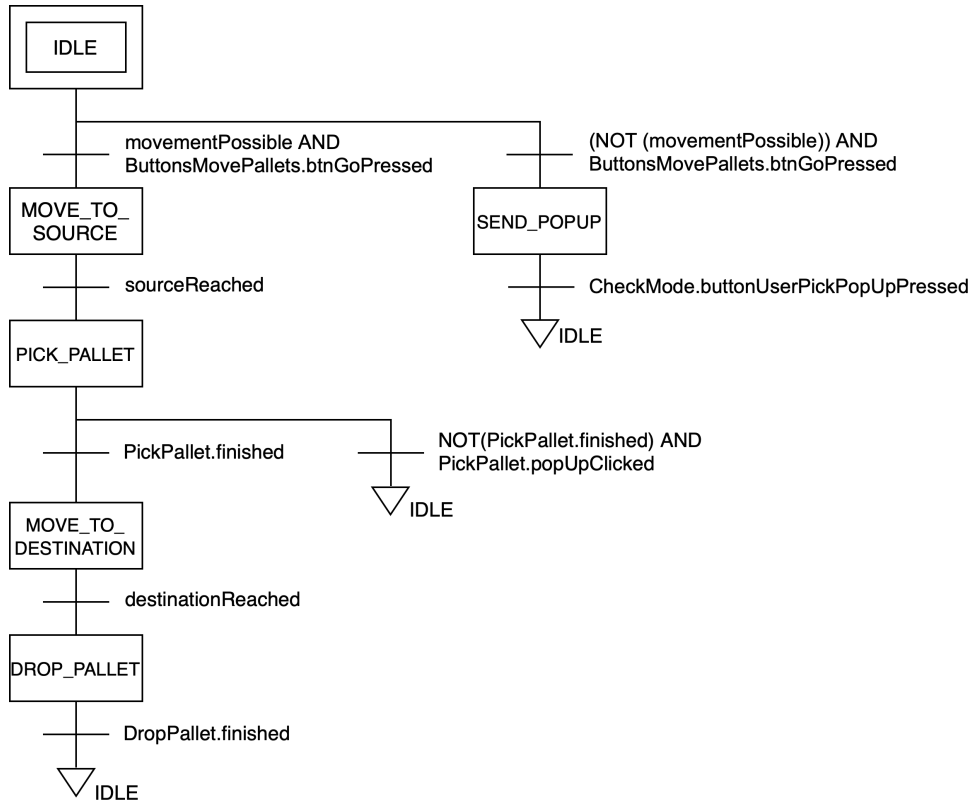


Figure 4.2: PickAndDrop SFC

The SFC executes through the following states:

- **IDLE**: Initializes the function, setting control flags and converting `sourceSlot` and `destinationSlot` into `sourcePos` and `destinationPos` (2D coordinates). It verifies the validity of input slots (not zero) and confirms that the source has a pallet and the destination is empty. If conditions allow, `movementPossible` is set to `TRUE`, enabling transition.

```

1  (* ----- IDLE ----- *)
2  CreateFlatSlotArray();
3  finished := FALSE;
4  showPopUp := FALSE;
5  movementPossible := FALSE;
6  sourcePos.X := sourceSlot.Row;
7  sourcePos.Z := (sourceSlot.Col - 1) * 2 + 1;
8  destinationPos.X := destinationSlot.Row;
9  destinationPos.Z := (destinationSlot.Col - 1) * 2 + 2;
10 IF NOT (sourceSlot.Row=0) AND NOT (sourceSlot.Col=0) AND NOT (destinationSlot
    .Row=0) AND NOT (destinationSlot.Col=0) THEN
11     IF NOT slot[sourceSlot.Row, sourceSlot.Col].isEmpty AND slot[
        destinationSlot.Row, destinationSlot.Col].isEmpty THEN
12         movementPossible := TRUE;
13     END_IF
14 END_IF
  
```



- **MOVE\_TO\_SOURCE**: Moves the mechanical arm to the `sourcePos` using `MoveXZ`. Upon reaching the source, `sourceReached` is set to `TRUE`.

```

1  (* ----- MOVE_TO_SOURCE ----- *)
2  IF NOT MoveXZ(sourcePos) THEN
3      sourceReached := TRUE;
4      destinationReached := FALSE;
5  END_IF

```

- **SEND\_POPUP**: If `PickPallet` indicates a pallet was not found, this state is entered to display a pop-up message to the user (`showPopUp := TRUE`). The system returns to `IDLE` once the pop-up interaction is complete.

```

1  (* ----- SEND_POPUP ----- *)
2  showPopUp := TRUE;

```

- **PICK\_PALLET**: Calls the `PickPallet` function block to execute the pallet retrieval sequence. The SFC transitions upon `PickPallet.finished` (successful pick) or if `PickPallet.popUpClicked` (pallet not found, pop-up sent).

```

1  (* ----- PICK_PALLET ----- *)
2  PickPallet();

```

- **MOVE\_TO\_DESTINATION**: Moves the mechanical arm to the `destinationPos` using `MoveXZ`. Once the destination is reached, `destinationReached` is set to `TRUE`.

```

1  (* ----- MOVE_TO_DESTINATION ----- *)
2  IF NOT MoveXZ(destinationPos) THEN
3      sourceReached := FALSE;
4      destinationReached := TRUE;
5  END_IF

```

- **DROP\_PALLET**: Calls the `DropPallet` function block to execute the pallet placement sequence. Upon `DropPallet.finished`, the `movementPossible` flag is reset, `finished` is set to `TRUE`, the pallet's data is transferred from the source to the destination slot using `CopySlotToAnother`, and the `sourceSlot` is cleared.

```

1  (* ----- DROP_PALLET ----- *)
2  DropPallet();
3  IF DropPallet.finished THEN
4      movementPossible := FALSE;
5      finished := TRUE;
6      (* Set the slot as occupied *)
7      CopySlotToAnother(slotOld:=sourceSlot, slotNew:=destinationSlot);
8      ClearSlot(slotToClear:=sourceSlot);
9  END_IF

```

## 5 | Store Pallet Mode

The Store Pallet Mode enables the system to receive a new pallet from the user and automatically store it in an optimal location within the warehouse. The process involves inputting a pallet ID, verifying warehouse capacity and ID uniqueness, and then orchestrating the mechanical arm's movements to load the pallet and place it into the nearest available slot.

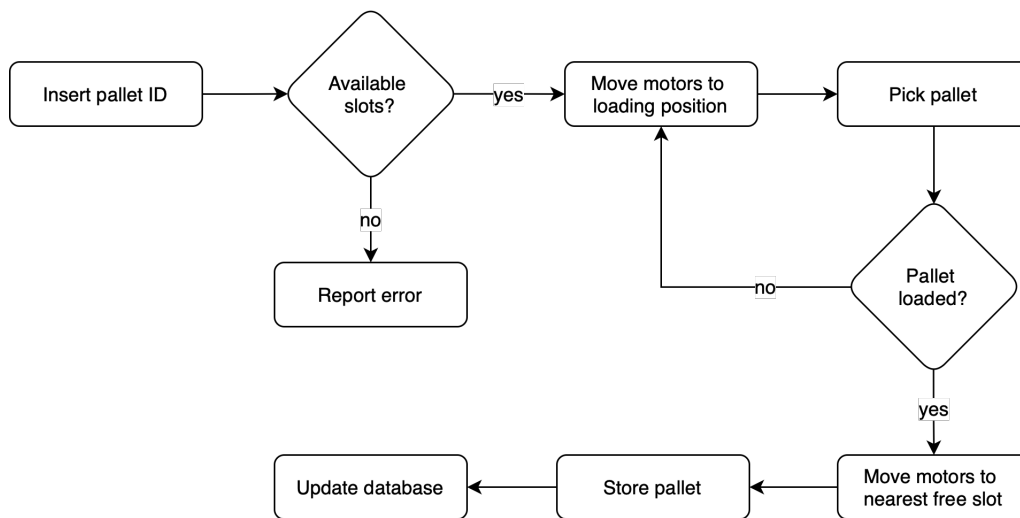


Figure 5.1: Store Pallet Mode Flowchart

As depicted in Figure 5.1, the system first checks for available slots and prompts for a pallet ID. If valid, it guides the user for manual loading before picking and storing the pallet in the optimal nearest slot and updating internal records.

The core logic for this mode is encapsulated within the `UserStorePallet` function block.

```

1 FUNCTION_BLOCK UserStorePallet
2 VAR
3     destinationSlot: SlotStruct;
4     movementPossible: BOOL := FALSE;
5     strPalletIDinStock: BOOL := FALSE;
6     homeReached: BOOL := FALSE;
7     destinationReached: BOOL := FALSE;
8     palletDetected: BOOL := FALSE;
9     finished: BOOL := FALSE;
10    state: INT;
  
```

```

11  strPalletID: STRING;
12  showPopupWarehouseFull: BOOL := FALSE;
13  showPopupPalletIDPresent: BOOL := FALSE;
14  strPalletIDIsEmpty: BOOL;
15  showPopupStringEmpty: BOOL := FALSE;
16  popupPalletIDContinuePressed: BOOL;
17  popupPalletIDStopPressed: BOOL;
18  DropPallet: DropPallet;
19  destinationPos: Pos2DStruct;
20  END_VAR

```

The detailed operational sequence of UserStorePallet is governed by its SFC, illustrated in Figure 5.2.

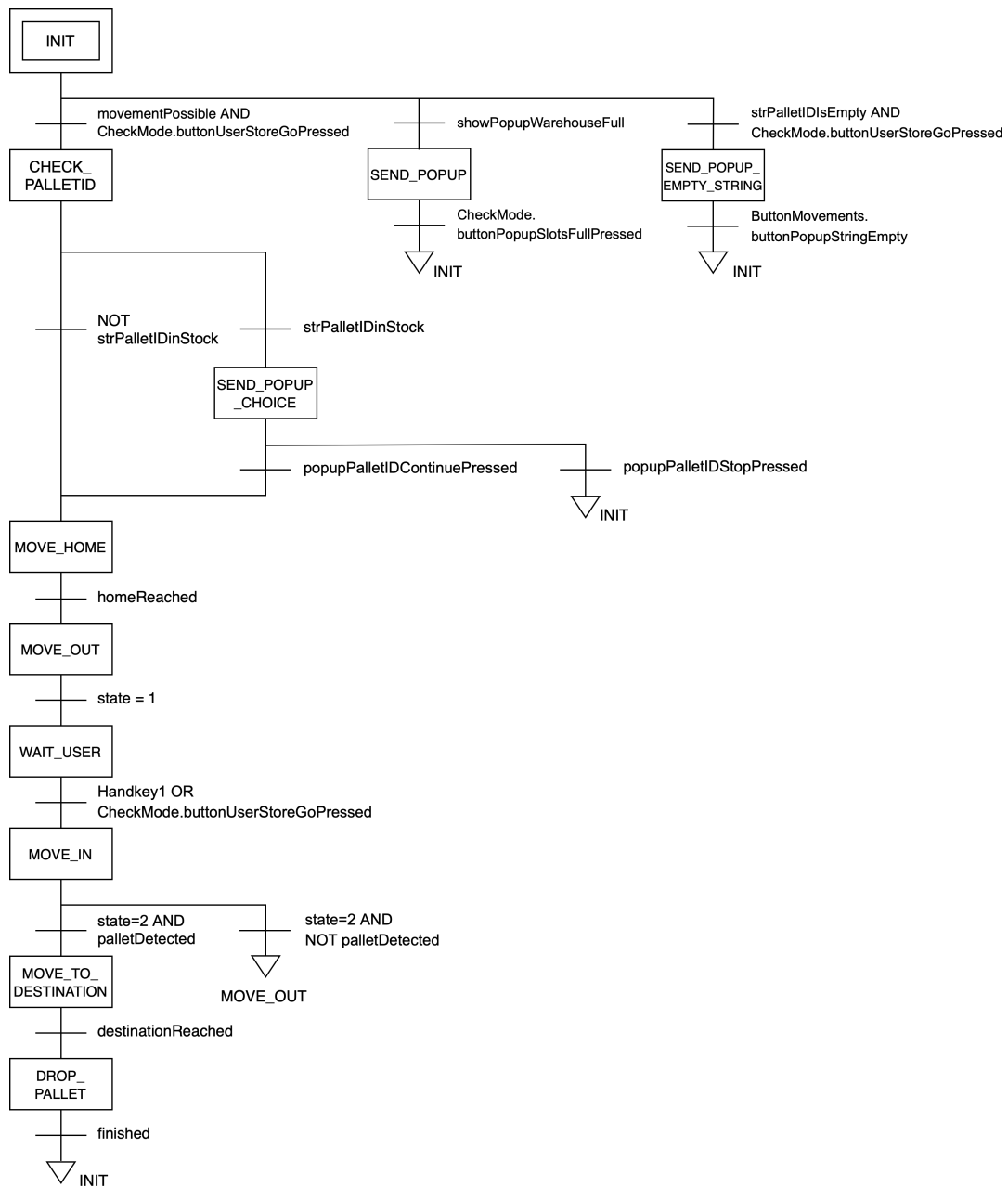


Figure 5.2: UserStorePallet SFC

The SFC transitions through the following key states:

- **INIT**: Initializes the function, determining the `destinationSlot` using `GetEmptySlot()` (which finds the nearest slot based on Manhattan distance). It also performs initial checks: if the warehouse is full (`SlotsFull()`), if the input `strPalletID` is empty, or if `strPalletID` already exists (`PalletIDInStock()`). Based on these checks, `movementPossible` is set, and relevant pop-ups are triggered.

```

1  (* ----- INIT ----- *)
2  destinationSlot := GetEmptySlot();
3  CreateFlatSlotArray();
4  showPopupWarehouseFull := SlotsFull();
5  IF SetPalletValue.str = '' THEN
6      strPalletIDIsEmpty:=TRUE;
7      movementPossible := FALSE;
8  ELSE
9      strPalletIDIsEmpty:=FALSE;
10     movementPossible := FALSE;
11 END_IF
12 IF destinationSlot.isEmpty AND (NOT(SetPalletValue.str='')) THEN
13     movementPossible := TRUE;
14     strPalletID := SetPalletValue.str;
15 END_IF
16 destinationPos.X := destinationSlot.Row;
17 destinationPos.Z := (destinationSlot.Col - 1) * 2 + 2;
18 finished := FALSE; destinationReached := FALSE;
19 homeReached := FALSE; palletDetected := FALSE;
20 showPopupPalletIDPresent := FALSE; strPalletIDInStock := FALSE;
21 popupPalletIDContinuePressed:= FALSE; popupPalletIDStopPressed := FALSE;
22 showPopupStringEmpty := FALSE;

```

- **CHECK\_PALLETID**: Verifies if the `strPalletID` (entered by the user) is already present in the warehouse using `PalletIDInStock()`.

```

1  (*---- CHECK_PALLETID --- *)
2  strPalletIDInStock := PalletIDInStock(strPalletID:=strPalletID);

```

- **SEND\_POPUP\_EMPTY\_STRING**: Displays a pop-up (`showPopupStringEmpty:=TRUE`) indicating that the pallet ID string is empty. The SFC returns to INIT from here if `CheckMode.buttonUserStoreGoPressed` is pressed.

```

1  (* --- SEND_POPUP_EMPTY_STRING ---*)
2  showPopupStringEmpty := TRUE;

```

- **SEND\_POPUP\_CHOICE**: Displays a pop-up (`showPopupPalletIDPresent := TRUE`) to the user, typically when a duplicate pallet ID is detected. It awaits user confirmation (continue) or cancellation (stop).

```

1  (* --- SEND_POPUP_CHOICE --- *)
2  showPopupPalletIDPresent := TRUE;
3  IF CheckMode.buttonPopupStorePalletIDContinuePressed THEN
4      showPopupPalletIDPresent := FALSE; popupPalletIDContinuePressed := TRUE;

```

```

5 END_IF
6 IF CheckMode.buttonPopupStorePalletIDStopPressed THEN
7     showPopupPalletIDPresent := FALSE; popupPalletIDStopPressed := TRUE;
8 END_IF

```

- **MOVE\_HOME:** Moves the mechanical arm to its home position using MoveHome(). homeReached is set to TRUE upon completion.

```

1 (* ----- MOVE_HOME ----- *)
2 showPopupPalletIDPresent:=FALSE;
3 IF NOT MoveHome() THEN
4     homeReached := TRUE;
5 END_IF

```

- **MOVE\_OUT:** Moves the stacker arm outwards (MoveY(3)) to allow manual pallet loading.

```

1 (* ----- MOVE_OUT ----- *)
2 IF NOT MoveY(3) THEN
3     state := 1;
4 END_IF

```

- **WAIT\_USER:** (Implicitly) waits for the user to manually load the pallet, transitioning based on Handkey1 or CheckMode.buttonUserStoreGoPressed.

```

1 (* --- This block is empty, waiting for external trigger --- *)

```

- **MOVE\_IN:** Moves the stacker arm inwards (MoveY(2)). If FeederOccupied is detected, palletDetected is set to TRUE.

```

1 (* ----- MOVE_IN ----- *)
2 IF FeederOccupied THEN
3     palletDetected := TRUE;
4 END_IF
5 IF NOT MoveY(2) THEN
6     state := 2;
7 END_IF

```

- **MOVE\_TO\_DESTINATION:** Moves the arm with the pallet to the determined destinationPos using MoveXZ. destinationReached is set TRUE on arrival.

```

1 (* ----- MOVE_TO_DESTINATION ----- *)
2 IF NOT MoveXZ(destinationPos) THEN
3     destinationReached := TRUE;
4 END_IF

```

- **DROP\_PALLET**: Calls the DropPallet function block to place the pallet into the destinationSlot. Upon DropPallet.finished, the destinationSlot's isEmpty status is updated to FALSE, and its palletID is set to strPalletID.

```
1  (* ----- DROP_PALLET ----- *)
2  DropPallet();
3  slot[destinationSlot.Row, destinationSlot.Col].isEmpty := FALSE;
4  slot[destinationSlot.Row, destinationSlot.Col].palletID := strPalletID;
5  IF DropPallet.finished THEN
6      movementPossible := FALSE;
7      finished := TRUE;
8  END_IF
```

## 6 | Pick Pallet Mode

The Pick Pallet Mode allows users to retrieve a specific pallet by simply entering its ID. The system automatically locates the pallet within the warehouse, moves the mechanical arm to that position, picks up the pallet, and returns it to the home position. This design ensures high flexibility, as the home position can be reconfigured to any location without impacting core system behavior, adapting to various warehouse layouts.

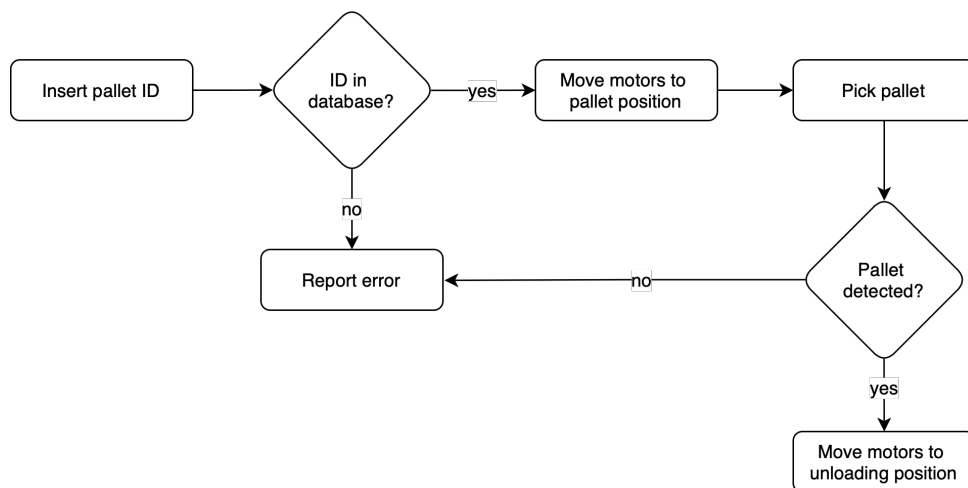


Figure 6.1: Pick Pallet Mode Flowchart

As outlined in Figure 6.1, the process involves ID input, pallet location, movement, picking, and database update. This entire sequence is managed by the `UserPickPallet` function block.

```

1 FUNCTION_BLOCK UserPickPallet
2 VAR
3   strPalletID: STRING;
4   destinationSlot: SlotStruct;
5   movementPossible: BOOL;
6   destinationReached: BOOL;
7   movedHome: BOOL;
8   finished: BOOL;
9   showPopUp: BOOL;
10  state: INT;
11  i,j: INT;
12  destinationPos: Pos2DStruct;

```

```

13 PickPallet: PickPallet;
14 END_VAR

```

The operational flow of UserPickPallet is defined by its SFC, depicted in 6.2.

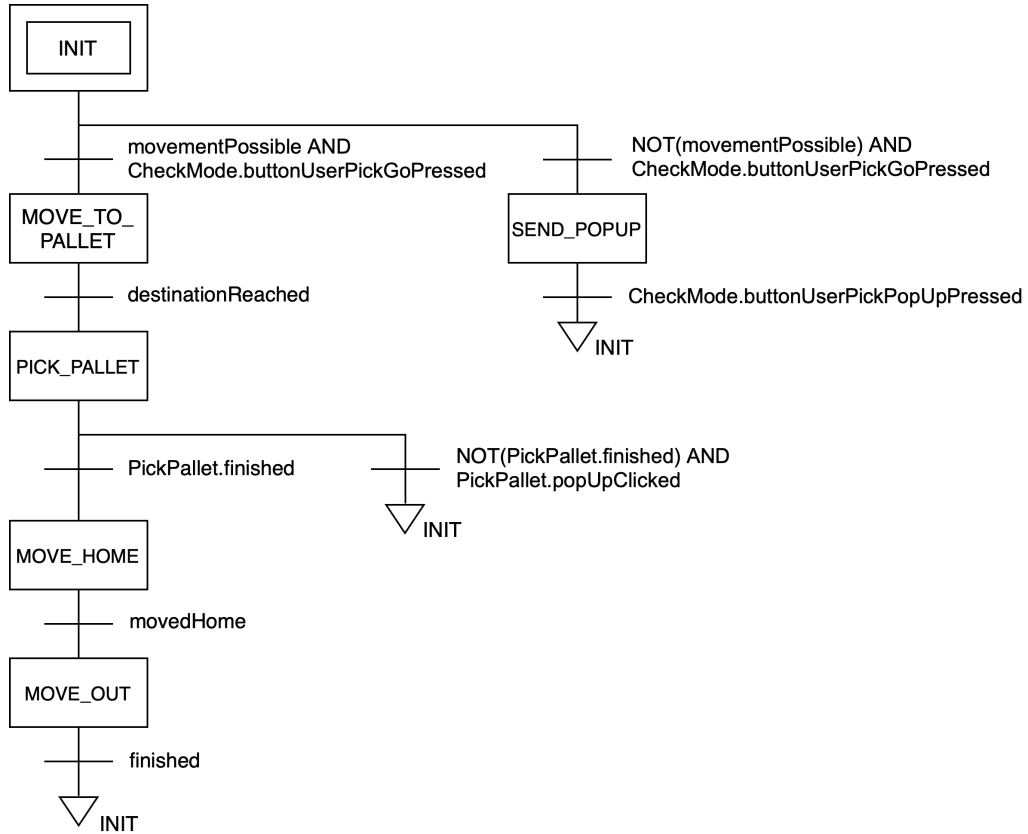


Figure 6.2: UserPickPallet SFC

The SFC transitions through the following states:

- **INIT**: Initializes the function. It retrieves the pallet ID from SetPalletValue.str and uses GetSlotFromPalletID to find its destinationSlot. If the pallet ID is valid and found, movementPossible is set to TRUE, and the destinationPos coordinates are determined.

```

1  (* ----- INIT ----- *)
2  movementPossible := FALSE; finished := FALSE; showPopUp := FALSE;
3  CreateFlatSlotArray();
4  IF (NOT(SetPalletValue.str='')) THEN
5      strPalletID := SetPalletValue.str;
6      destinationSlot := GetSlotFromPalletID(strPalletID:=strPalletID);
7      IF destinationSlot.palletID=strPalletID THEN
8          movementPossible := TRUE;
9      END_IF
10     destinationPos.X := destinationSlot.Row;
11     destinationPos.Z := (destinationSlot.Col - 1) * 2 + 1;
12 END_IF

```



- **MOVE\_TO\_PALLET:** Commands the mechanical arm to move to the destinationPos using MoveXZ. destinationReached is set TRUE upon arrival.

```

1  (* ----- MOVE_TO_PALLET ----- *)
2  IF NOT MoveXZ(destinationPos) THEN
3      destinationReached := TRUE;
4  ELSE
5      destinationReached := FALSE;
6  END_IF

```

- **PICK\_PALLET:** Calls the PickPallet function block to execute the pallet retrieval sequence.

```

1  (* ----- PICK_PALLET ----- *)
2  PickPallet();

```

- **MOVE\_HOME:** Moves the mechanical arm to its predefined home position using MoveHome(). movedHome is set TRUE once the arm is at home.

```

1  (* ----- MOVE_HOME ----- *)
2  IF NOT MoveHome() THEN
3      movedHome := TRUE;
4  ELSE
5      movedHome := FALSE;
6  END_IF

```

- **MOVE\_OUT:** Moves the arm outwards (MoveY(3)). Upon completion, finished is set to TRUE, and the destinationSlot (where the pallet was picked from) is cleared using ClearSlot.

```

1  (* ----- MOVE_OUT ----- *)
2  IF NOT MoveY(3) THEN
3      finished := TRUE;
4  ELSE
5      finished := FALSE;
6  END_IF
7  ClearSlot(slotToClear:=destinationSlot);

```

# 7 | Graphical User Interface

The graphical interface for managing the system was developed using ABB Automation Builder. It was decided to structure the interface starting from the home screen and then divide the various functions into five different screens, which will be discussed individually below. Each interface is equipped with an emergency button at the top right that stops the entire system; at the top left is the button to return to the home screen, while the system operating status is shown at the bottom left. Any messages or errors are reported to the user in the form of pop-ups. The graphic choices implemented have been designed to scale along with the warehouse and adapt to any number of rows and columns.

## 7.1 Mode selection interface

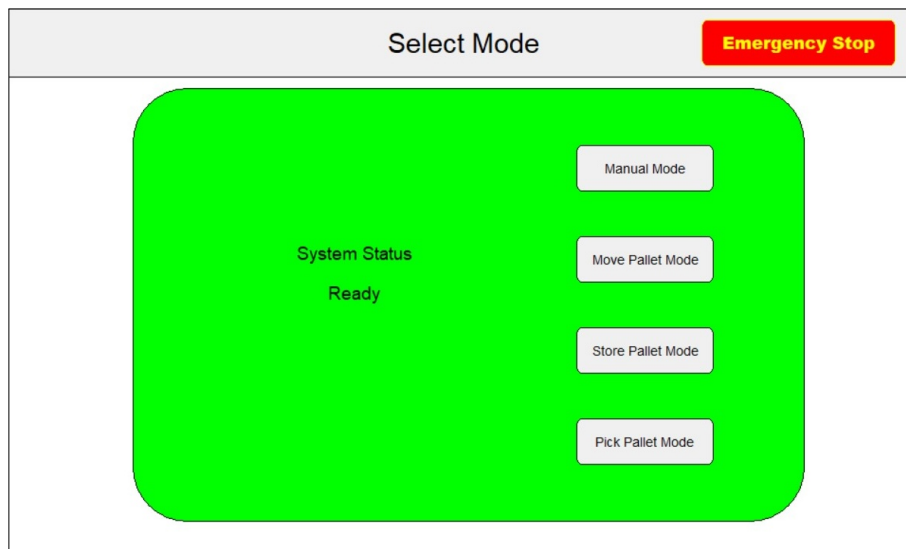


Figure 7.1: Homepage

When the system is turned on, the home page is displayed, from which it is possible to choose one of the four operating modes of the system: Manual, Move Pallet, Store Pallet or Pick Pallet. Under normal conditions, the background is green to indicate that the system is ready; in case of errors, the screens shown in Figure 7.2 and Figure 7.3 are displayed.

In the first case, the background turns red when the emergency button is pressed, which can be reset to normal by holding down the physical button for 5 seconds. In the second case, instead, the background turns purple when the system detects that multiple switches have been pressed simultaneously on the same axis.

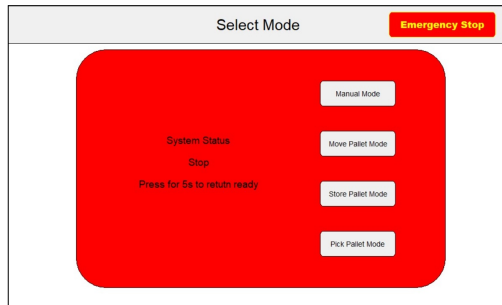


Figure 7.2: Emergency stop

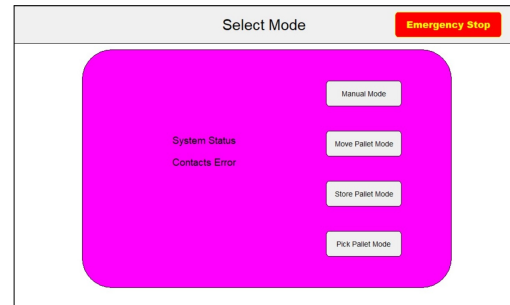


Figure 7.3: Contacts error

## 7.2 Manual mode interface

In manual mode, each axis can be moved individually. The three axes are shown intuitively on the left, while the current coordinates of the system are shown on the right. The button below allows you to access the database management mode, which will be discussed in the next section. During axis movement, the corresponding button is displayed in green (Figure 7.5). whereas the squares on each axis represent the individual switches, providing visual feedback on the position of the arm; in any case, it is not possible to exceed the extreme coordinates of each axis.

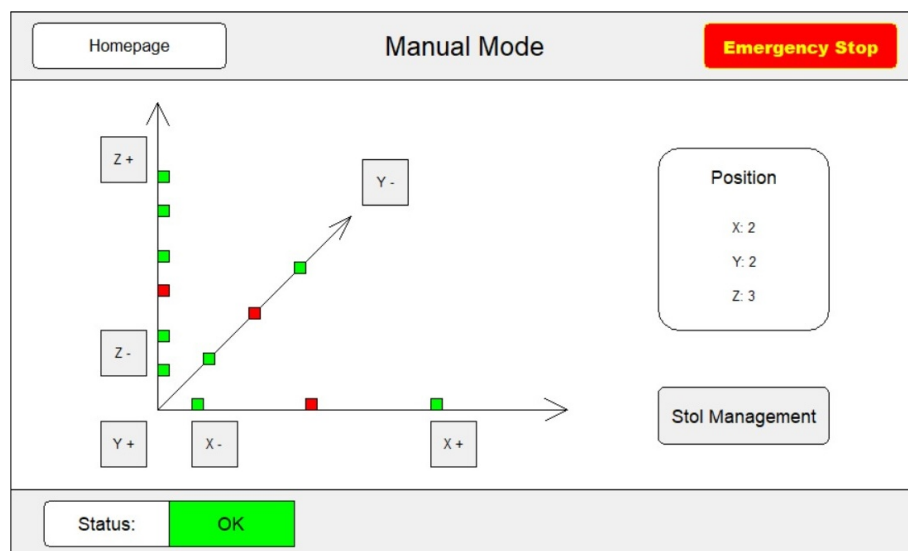


Figure 7.4: Manual mode

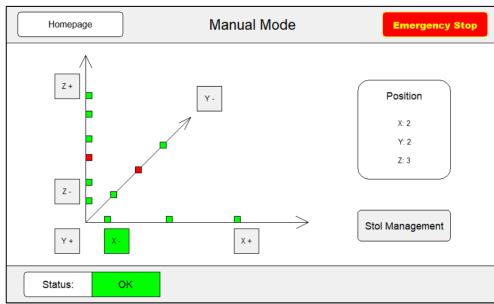


Figure 7.5: Moving axis

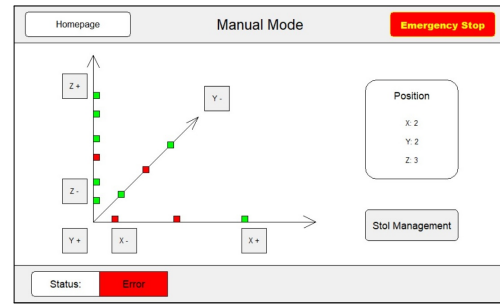


Figure 7.6: Multiple switches pressed

## 7.3 Slot management interface

The slot management interface allows the user to manually edit the warehouse database. The table shows the current status of the warehouse with all pallets saved in memory; using the interface on the left, it is possible to add a pallet to a specific position by assigning it a unique ID or remove it if its ID is known.

If the user attempts to insert a pallet into a slot that is already occupied or attempts to remove a pallet that does not exist, a pop-up window will appear notifying him of the error.

| Row | Col | palletID |
|-----|-----|----------|
| 1   | 1   |          |
| 1   | 2   |          |
| 1   | 3   |          |
| 2   | 1   |          |
| 2   | 2   |          |
| 2   | 3   |          |
| 3   | 1   | Pallet 1 |
| 3   | 2   |          |
| 3   | 3   | Pallet 2 |
|     |     |          |

Figure 7.7: Slot management mode

## 7.4 Pick pallet interface

In the pick pallet screen, it is possible to enter the ID of the pallet to be picked. The table can be used to search for pallets currently in stock. Again, if a non-existent ID is entered, the window shown in Figure 7.9 will appear. On the other hand, if the pick command is executed but the sensor does not detect any pallets being picked

up, the pop-up shown in Figure 7.10 is displayed. However, this warning can be ignored and the command can still be completed by pressing the continue button.

| Row | Col | palletID |
|-----|-----|----------|
| 1   | 1   |          |
| 1   | 2   |          |
| 1   | 3   |          |
| 2   | 1   |          |
| 2   | 2   |          |
| 2   | 3   |          |
| 3   | 1   | Pallet 1 |
| 3   | 2   |          |
| 3   | 3   | Pallet 2 |

Figure 7.8: Pick Pallet Mode

Figure 7.9: Nonexistent pallet

Figure 7.10: Missing pallet

## 7.5 Store pallet interface

The interface for storing pallets requires the entry of a unique ID; the string can be entered using the keyboard or, if there is a touchscreen display, using the pad shown in Figure 7.11. When the “Store” button is pressed, the system will place the pallet in the nearest free slot and will update the database table. Clearly, before any movement, a check is performed to ensure that the string entered is valid. In the case the entered string is already present in the warehouse, a choice-window is shown, and the user can store the pallet anyway (Figure 7.13). If the warehouse is full, the entry interface is disabled and the only option is to return to the homepage, as shown in Figure 7.12.

| Row | Col | palletID |
|-----|-----|----------|
| 1   | 1   |          |
| 1   | 2   |          |
| 1   | 3   |          |
| 2   | 1   |          |
| 2   | 2   |          |
| 2   | 3   |          |
| 3   | 1   | Pallet 1 |
| 3   | 2   |          |
| 3   | 3   | Pallet 2 |

Figure 7.11: Store Pallet Mode

| Row | Col | palletID |
|-----|-----|----------|
| 1   | 1   | Pallet 4 |
| 1   | 2   | Pallet 6 |
| 1   | 3   | Pallet 5 |
| 2   | 1   | Pallet 8 |
| 2   | 2   | Pallet 7 |
| 2   | 3   | Pallet 9 |
| 3   | 1   | Pallet 1 |
| 3   | 2   | Pallet 3 |
| 3   | 3   | Pallet 2 |

Figure 7.12: Full warehouse

Figure 7.13: palletID already present

## 7.6 Move pallet interface

The interface for storing pallets requires the entry of a unique ID, which can be entered via keyboard or, on touchscreen displays, using the numeric pad (Figure 7.15). Upon pressing the "Store" button, the system attempts to place the pallet in the nearest free slot and updates the database. Crucially, before any movement, the entered string is validated. If a pallet with the same ID already exists, a pop-up allows the user to either "Continue," forcing the insertion and enabling multiple pallets with the same ID to be stored, or "Back" to cancel. Additionally, if the warehouse is full (Figure 7.16), the entry interface is disabled, restricting options to returning to the homepage.

Similarly to the pick pallet interface, if the hall-effect sensor does not detect the presence of the pallet, the user is notified with an error message that can be ignored by clicking "Continue" and completing the command, or managed by interrupting the operation.

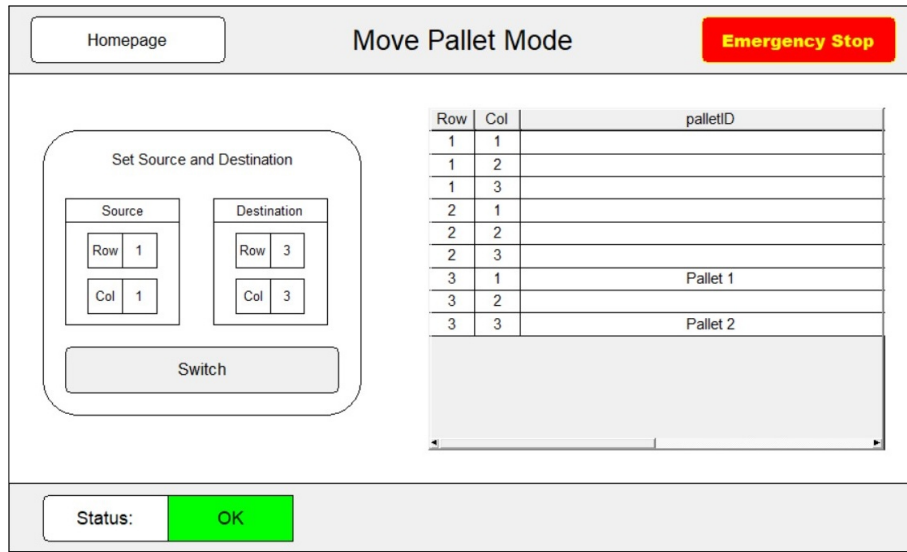


Figure 7.14: Move Pallet Mode

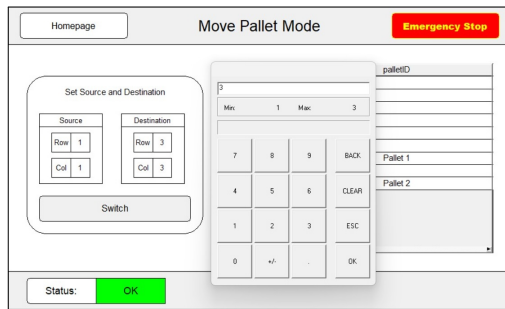


Figure 7.15: Keypad

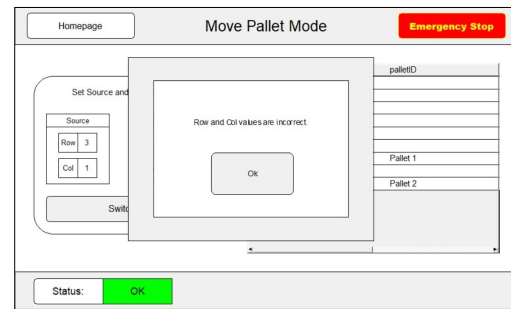


Figure 7.16: Invalid row or col value

An additional useful feature that could be integrated into our software is the implementation of communication with an external database. This functionality would enable the system to store the current state of the grid externally, ensuring data persistence even in the event of system restarts or unexpected shutdowns. Furthermore, the ability to automatically retrieve and reload the saved grid configuration at startup would significantly improve usability and reliability, especially in long-term or continuous operation scenarios. Such an enhancement would also facilitate remote monitoring, diagnostics, and potentially the integration with higher-level enterprise systems.

We attempted to implement a simplified version of this functionality using the `SysLibFile` library. Although the code compiled successfully and was free of errors, the implementation did not produce the expected results due to a file permission issue. In particular, the line `hFile := SysFileOpen(filePath, 'w')`, that manages the opening (and creation if the file does not exist) returns always 0, which corresponds to the system is unable to manage the file error. Due to the fact that this error occur both in the real PLC and in Simulation Mode, we believe that CoDeSys 2.3 restricts the file management.

# 8 | Scalability

In this chapter, we will focus on how to increase (or decrease) the size of the warehouse.

## 8.1 Number of Rows and Columns

By recalling the global constants, it is possible to modify the following four constants to change the size.

```
1  VAR_GLOBAL CONSTANT
2    N_ROWS : INT := 3;
3    N_COLS : INT := 4; (* Add a new col *)
4    N_SLOTS : INT := 12;
5    N_Z_SENSORS : INT := 6; (*Do 2*N_ROWS*)
6    [...]
```

The user is needed to manually calculate  $N\_SLOTS = N\_COLS * N\_COLS$  and  $N\_Z\_SENSORS = 2 * N\_ROWS$  because CoDeSys 2.3 does not allow algebraic operation in the phase of declaration of vectors.

## 8.2 Add Sensors

New input signals for movement sensor must be connected to the `xPosSensors` and `zPosSensors` array in physical order.

```
1  FUNCTION UpdateArraySensors : BOOL
2    [...]
3    xPosSensors[3] := XposRight;
4    xPosSensors[4] := XposRightR; (* Add a new col *)
5    [...]
```

## 8.3 Visualization

The variable responsible for the representation of the `slot` matrix in the GUI is the function `CreateFlatSlotArray`, which converts the 2-dimension matrix into an array because CoDeSys 2.3 can elaborate only 1-dimension array in the GUI.

By modifying the dimension in `VAR_GLOBAL CONSTANT`, the table increases its dimension automatically.



## 9 | Testing and Validation

The system underwent a comprehensive functional verification phase, which was structured into multiple operational scenarios in order to validate the logical behavior of the machine. The testing activities were carefully designed to cover both standard operating conditions and potential edge cases. Specifically, the following functionalities were tested:

- **Manual Movements:** The response of the motors along all three axes (X, Y, and Z) was tested to ensure accurate movement within the defined physical boundaries. Limit switches were used to verify that the system appropriately halts motion when reaching mechanical constraints.
- **Move Pallet Mode:** The system was tested for its ability to transfer pallets between valid slots. Tests included scenarios where the source slot was occupied and the destination slot was free, ensuring that the logic governing these transitions was correctly implemented.
- **Store and Pick Pallet Mode:** The insertion and retrieval of pallets by ID were tested, including both typical use cases and edge conditions such as duplicate IDs, full storage conditions, and attempts to retrieve non-existent pallet IDs. These tests confirmed the robustness of the ID management system.
- **Error Handling:** The system's ability to detect and respond to errors was evaluated. This included verification of pop-up alert messages, LED status indications, and motor shutdown procedures in the event of abnormal sensor readings (e.g., multiple sensors active simultaneously) or sensor disconnections.
- **Scalability:** The system's adaptability to varying warehouse dimensions was assessed, with successful implementations for 2x2 and 1x3 configurations. While the software design supports broader scalability, physical limitations of the prototype system prevented testing beyond a 3x3 grid.

The results of the functional tests confirmed that the system behaves correctly across all intended operational modes, including under simulated fault conditions. Additionally, the modular structure of the control code facilitated quick identification and correction of any anomalies encountered during testing, thereby improving system reliability and maintainability.

# 10 | Conclusion

The aim of this project was to automate a scaled warehouse model using a PLC and the CoDeSys development environment. The final system features four operating modes: manual, move, store and pick. Each mode is accessible through a structured and user-friendly graphical interface, designed to guide the user through all functions.

The control logic was built on a modular architecture that ensures scalability. All parameters, such as grid size and sensor layout were defined through global constants, making it easy to adapt the system to different warehouse dimensions or hardware configurations. The use of Structured Text and Sequential Function Charts enabled clear organization of operations, from axis movement to slot management.

Safety was a central aspect of the design. The system continuously verifies the status of position sensors and disables motor movement in case of inconsistent or unsafe readings. Error messages help the user to interact with the system effectively.

All core functionalities—including manual control, pallet storage and retrieval, and system feedback—were thoroughly tested under realistic conditions. These tests demonstrated that the system can operate effectively, even when subject to common edge cases or user errors.

Overall, the project delivers a complete and flexible automation solution for warehouse systems, demonstrating a solid application of control logic, safety principles, and user interface design.

# List of Figures

|      |                                       |    |
|------|---------------------------------------|----|
| 1.1  | Warehouse Model . . . . .             | 4  |
| 1.2  | Pin Mapping . . . . .                 | 5  |
| 1.3  | PLC inputs . . . . .                  | 6  |
| 1.4  | PLC outputs . . . . .                 | 7  |
| 2.1  | DropPallet SFC . . . . .              | 22 |
| 2.2  | PickPallet SFC . . . . .              | 24 |
| 3.1  | Manual mode flowchart . . . . .       | 26 |
| 4.1  | Move Pallet Mode Flowchart . . . . .  | 29 |
| 4.2  | PickAndDrop SFC . . . . .             | 31 |
| 5.1  | Store Pallet Mode Flowchart . . . . . | 33 |
| 5.2  | UserStorePallet SFC . . . . .         | 34 |
| 6.1  | Pick Pallet Mode Flowchart . . . . .  | 38 |
| 6.2  | UserPickPallet SFC . . . . .          | 39 |
| 7.1  | Homepage . . . . .                    | 41 |
| 7.2  | Emergency stop . . . . .              | 42 |
| 7.3  | Contacts error . . . . .              | 42 |
| 7.4  | Manual mode . . . . .                 | 42 |
| 7.5  | Moving axis . . . . .                 | 43 |
| 7.6  | Multiple switches pressed . . . . .   | 43 |
| 7.7  | Slot management mode . . . . .        | 43 |
| 7.8  | Pick Pallet Mode . . . . .            | 44 |
| 7.9  | Nonexistent pallet . . . . .          | 44 |
| 7.10 | Missing pallet . . . . .              | 44 |
| 7.11 | Store Pallet Mode . . . . .           | 45 |
| 7.12 | Full warehouse . . . . .              | 45 |
| 7.13 | palletID already present . . . . .    | 45 |
| 7.14 | Move Pallet Mode . . . . .            | 46 |
| 7.15 | Keypad . . . . .                      | 46 |
| 7.16 | Invalid row or col value . . . . .    | 46 |